



المخبرية الوطنية العليا للفنون والمهن بالرباط
Hoches Écoles Nationales d'Arts et Métiers de Rabat
Ecole Nationale Supérieure d'Arts et Métiers de Rabat



ROYAUME DU MAROC

UNIVERSITÉ MOHAMMED V DE RABAT

École Nationale Supérieure d'Arts et Métiers de Rabat

(ENSAM RABAT)

**Filière : Ingénierie des Données et Intelligence
Artificielle**

(INDIA – SD)

RAPPORT DE STAGE D'INITIATION

**Conception et Implémentation
d'un Backend RAG Hybride
pour la Recherche et la Consultation
des Procédures Administratives**

Réalisé par :

Lahsaini Yazid

Encadré par :

Mr Fadili Abdelkader

(Encadrant professionnel – MTNRA)

Pr Mourad El Yadari

(Encadrant académique – ENSAM Rabat)

Organisme d'accueil :

Ministère de la Transition Numérique et de la Réforme de
l'Administration
(MTNRA)

Année Universitaire : 2025 – 2026

Remerciements

Au terme de ce stage d'initiation, je tiens à exprimer ma profonde gratitude et mes plus sincères remerciements à toutes les personnes qui, par leur soutien, leurs conseils et leur confiance, ont contribué à la réussite de ce travail portant sur la conception et l'implémentation d'un backend RAG hybride pour les services publics numériques marocains.

Je souhaite tout d'abord remercier le **Ministère de la Transition Numérique et de la Réforme de l'Administration (MTNRA)** pour m'avoir accueilli au sein de ses équipes et offert l'opportunité de réaliser ce stage d'initiation dans un cadre de travail stimulant et propice à l'apprentissage. La confiance qui m'a été accordée pour concevoir et développer un système backend RAG appliqué aux services publics numériques a constitué une expérience professionnelle particulièrement enrichissante, tant sur le plan technique qu'humain.

Mes remerciements les plus sincères vont également à mon **encadrant professionnel** au sein du Ministère, pour son orientation technique avisée, sa disponibilité constante et ses conseils précieux tout au long de ce stage. Son suivi régulier et ses remarques pertinentes ont grandement contribué à orienter mes travaux et à en garantir la pertinence et la rigueur.

Je tiens par ailleurs à exprimer ma reconnaissance envers l'ensemble de **l'équipe pédagogique de l'École Nationale Supérieure des Arts et Métiers (ENSAM) de Rabat**, et plus particulièrement les professeurs de la filière **Ingénierie des Données et Intelligence Artificielle (INDIA-SD)**, pour la qualité et l'excellence de la formation théorique et pratique qu'ils m'ont transmise durant mon parcours académique. Les compétences acquises grâce à leur enseignement ont constitué le socle indispensable à la réalisation de ce projet.

J'adresse également mes vifs remerciements à mon **encadrant académique**, pour son accompagnement attentif, ses remarques constructives et la rigueur méthodologique qu'il n'a cessé de m'inculquer tout au long de ce travail. Ses conseils avisés ont été déterminants dans la structuration et la qualité de ce rapport.

Je remercie chaleureusement les **membres du jury** pour l'honneur qu'ils me font en acceptant d'évaluer ce travail de fin de stage d'initiation. Leur lecture attentive et leurs observations constitueront pour moi un apport précieux.

Enfin, je ne saurais conclure ces remerciements sans exprimer ma gratitude envers mes **proches**, mes **camarades de promotion** ainsi que mes **collègues**, pour leur soutien moral et technique constant, leur écoute et leurs encouragements tout au long de cette expérience de stage.

À toutes et à tous, j'adresse l'expression de ma sincère et profonde reconnaissance.

Lahsaini Yazid

Table des matières

Remerciements	i
1 Contexte Général, Problématique Métier et Objectifs du Stage	1
1.1 Contexte général du stage	1
1.1.1 La digitalisation des services publics numériques marocains . . .	1
1.1.2 Un besoin d’interrogation intelligente du corpus administratif .	1
1.2 Problématique métier et technique	2
1.2.1 Problématique métier	2
1.2.2 Limites des grands modèles de langage classiques	2
1.2.3 Difficultés techniques associées	3
1.3 Objectifs fonctionnels et techniques du système RAG	4
1.4 Périmètre fonctionnel du projet	5
1.4.1 Deux sous-systèmes complémentaires	5
1.4.2 Évolution du corpus au cours du stage	5
1.4.3 Synthèse du positionnement du projet	6
2 État de l’Art et Fondements Théoriques du Retrieval-Augmented Generation	7
2.1 Introduction	7
2.2 Le Paradigme RAG face au Fine-Tuning	7
2.2.1 Limites du LLM comme base de connaissance autonome	7
2.2.2 Deux approches concurrentes : Fine-Tuning vs RAG	8
2.2.3 Justification du choix RAG pour le corpus administratif	8
2.3 Recherche Lexicale Sparse : le Modèle BM25	9
2.3.1 Principe général	9
2.3.2 Formulation mathématique	9
2.3.3 Pertinence pour le corpus administratif marocain	10
2.4 Recherche Vectorielle Dense : Similarité Sémantique dans un Espace d’Embeddings	11
2.4.1 Principe des embeddings sémantiques	11
2.4.2 Similarité cosinus	11

2.4.3	Architecture Bi-Encoder et modèle multilingue retenu	12
2.5	Fusion Hybride Sparse-Dense	12
2.6	Reranking par Cross-Encoder	13
2.6.1	Limite structurelle des Bi-Encoders	13
2.6.2	Architecture Cross-Encoder	13
2.6.3	Architecture en deux étages : rappel puis précision	14
2.7	Query Expansion : Augmentation du Rappel par Reformulation	14
2.7.1	Motivation théorique	14
2.7.2	Implémentation retenue	15
2.7.3	Compromis rappel/latence/coût	15
2.8	Synthèse	15
3	Architecture Technique et Conception du Système RAG	17
3.1	Introduction	17
3.2	Pipeline Séquentiel End-to-End	17
3.2.1	Vue d'ensemble du flux de traitement	17
3.2.2	Description séquentielle des étapes	19
3.3	Stack Technologique Final	20
3.4	Évolution du Vector Store : de ChromaDB à MongoDB Atlas Vector Search	20
3.4.1	Contexte historique	20
3.4.2	Transition vers MongoDB Atlas Vector Search	20
3.4.3	Remarque sur la cohérence documentaire	22
3.4.4	Justification du choix final	22
3.5	Stratégie de Caching Exact avec Redis/Upstash	23
3.5.1	Principe général	23
3.5.2	Normalisation et génération de la clé	23
3.5.3	Lecture et écriture du cache	23
3.5.4	Invalidation du cache	24
3.5.5	Migration de Redis local vers Redis cloud	24
3.6	API REST FastAPI	25
3.6.1	Principes de conception de l'API	25
3.6.2	Validation des entrées avec Pydantic	25
3.6.3	Cartographie des endpoints	25
3.6.4	Exemple d'appel à l'endpoint principal	25
3.6.5	Gestion documentaire via CRUD	26
3.7	Synthèse du chapitre	27

4	Implémentation, Pipeline d’Ingestion et Synchronisation d’État	28
4.1	Introduction	28
4.2	Pipeline d’Ingestion Multi-Format	28
4.2.1	Vue d’ensemble	28
4.2.2	Extraction des documents Word (.docx)	29
4.2.3	Extraction des documents PDF (.pdf)	30
4.2.4	Extraction des fichiers Excel (.xls / .xlsx)	31
4.2.5	Stratégie de chunking : découpage fixe sans chevauchement . . .	32
4.2.6	Construction des objets <code>Document</code> <code>LangChain</code>	33
4.3	Le Problème de Désynchronisation d’État	34
4.3.1	Origine du problème	34
4.3.2	Solution retenue : <code>sync_retrievers()</code>	34
4.3.3	Flux de synchronisation	36
4.4	Gestion Dynamique du Cache lors des Opérations CRUD	36
4.4.1	Justification de l’invalidation systématique	36
4.4.2	Intégration dans les endpoints CRUD	37
4.4.3	Synthèse du couplage CRUD – État	38
4.5	Conteneurisation et Déploiement Docker	39
4.5.1	Objectifs de la conteneurisation	39
4.5.2	Analyse du Dockerfile	39
4.5.3	Discussion	41
4.6	Conclusion du Chapitre	42
5	Évaluation Expérimentale, Résultats et Analyse des Performances	43
5.1	Introduction et objectifs de l’évaluation	43
5.2	Protocole d’évaluation et métriques RAGAS	44
5.2.1	Choix du framework RAGAS	44
5.2.2	Résultats RAGAS documentés dans le README	44
5.2.3	Résultats RAGAS d’une configuration historique intermédiaire .	45
5.3	Profilage de la latence et décomposition temporelle	46
5.3.1	Méthodologie de mesure	46
5.3.2	Comparaison des versions v17 et v18	46
5.3.3	Décomposition proportionnelle de la latence	47
5.4	Analyse des compromis (Trade-offs)	48
5.4.1	Qualité du reranker versus latence	48
5.4.2	Impact de la Query Expansion sur les tokens et le rappel documentaire	49
5.4.3	Synthèse de l’arbitrage global	50
5.5	Rôle de la sélection des trois meilleurs chunks (Top-3)	50

5.5.1	Justification technique	51
5.5.2	Lien avec les résultats mesurés	51
5.6	Synthèse du chapitre	52
6	Limites Apparentes, Retours d'Expérience et Perspectives d'Évolution	53
6.1	Introduction	53
6.2	Limites Techniques Identifiées	53
6.2.1	Un cache exact, non sémantique	53
6.2.2	Reconstruction synchrone et intégrale de l'index BM25	54
6.2.3	Chunking fixe à 1000 mots sans chevauchement	55
6.2.4	Absence de citation des sources dans la réponse API	55
6.2.5	Synthèse des limites	56
6.3	Axes d'Amélioration et Perspectives Techniques	56
6.3.1	Mise en place d'un cache sémantique	56
6.3.2	Query expansion conditionnelle	56
6.3.3	Remplacement de la pondération linéaire par une fusion de rangs (RRF)	57
6.3.4	Vers un RAG agentique (Agentic RAG / Self-RAG)	58
6.3.5	Renforcement de la sécurité applicative	58
6.4	Bilan Personnel et Synthèse des Acquis du Stage	59

Table des figures

1.1	Évolution du corpus documentaire au cours du stage	6
-----	--	---

Liste des tableaux

1.1	Objectifs du système RAG et réalisation associée dans le projet	4
2.1	Comparaison Bi-Encoder vs Cross-Encoder	13
3.1	Stack technologique de la version finale v25	21
3.2	Endpoints REST exposés par le backend v25	26
4.1	Actions de synchronisation associées à chaque opération CRUD documentaire	39
5.1	Scores RAGAS documentés dans le README (résultats expérimentaux internes, jeu de test de développement)	45
5.2	Scores RAGAS d'une configuration historique intermédiaire (mesure expérimentale, non représentative de v25)	45
5.3	Comparaison des temps mesurés entre les versions v17 (sans expansion) et v18 (avec expansion), mesures expérimentales sur scénario de test . .	47
5.4	Part relative de chaque composant dans le temps total (calculée à partir du tableau 5.3)	47
5.5	Synthèse comparative des rerankers testés (observations expérimentales qualitatives)	48
5.6	Synthèse des compromis qualité / latence / coût token observés (mesures expérimentales internes)	50
6.1	Synthèse des limites techniques identifiées sur la version v25	56

Listings

3.1	Initialisation du vector store MongoDB Atlas	20
3.2	Normalisation et construction de la clé de cache	23
3.3	Lecture du cache avant exécution du pipeline	23
3.4	Écriture du cache avec TTL de 24 heures	24
3.5	Connexion au client Redis via Upstash	24
3.6	Exemple de requête sur l'endpoint <code>/ask</code>	25
3.7	Exemple de structure de réponse de l'endpoint <code>/ask</code>	26
4.1	Extraction du texte d'un fichier DOCX	29
4.2	Extraction du texte d'un fichier PDF	30
4.3	Extraction et linéarisation d'un fichier Excel	31
4.4	Génération de l'identifiant de chunk par hachage MD5	32
4.5	Construction des objets Document avec métadonnées	33
4.6	Fonction de synchronisation des retrievers après modification de la base	34
4.7	Séquence d'ajout d'un document et invalidation du cache	37
4.8	Suppression d'un document et invalidation du cache	37
4.9	Dockerfile de conteneurisation du backend RAG	39
5.1	Sélection finale des trois meilleurs chunks après reranking	51
6.1	Construction de la clé de cache	53
6.2	Reconstruction complète de l'index sparse	54
6.3	Découpage sans chevauchement dans la version finale	55

Chapitre 1

Contexte Général, Problématique Métier et Objectifs du Stage

1.1 Contexte général du stage

1.1.1 La digitalisation des services publics numériques marocains

Le Maroc s'inscrit depuis plusieurs années dans une dynamique de transformation numérique de son administration, visant à simplifier l'accès des citoyens aux procédures administratives et aux services publics. Cette digitalisation se traduit par la multiplication des plateformes en ligne, des portails d'information et des bases documentaires décrivant les démarches à suivre pour obtenir un document administratif, effectuer une inscription, ou accomplir une formalité réglementaire.

Cependant, cette abondance d'information numérique s'accompagne d'une difficulté nouvelle : la dispersion et l'hétérogénéité des sources documentaires. Un usager cherchant une information précise doit fréquemment naviguer entre plusieurs portails, documents PDF, fiches procédurales ou pages web, sans disposer d'un point d'entrée unique capable de répondre directement à sa question en langage naturel. C'est dans ce contexte que s'inscrit le stage, dont l'objectif a été de concevoir un système capable d'interroger un corpus documentaire administratif et de fournir une réponse contextualisée, fiable et traçable.

1.1.2 Un besoin d'interrogation intelligente du corpus administratif

Le stage a porté sur la conception et l'implémentation d'un système de question-réponse fondé sur le paradigme **Retrieval-Augmented Generation (RAG)**, appliqué à des informations relatives aux services publics numériques marocains et aux procédures

administratives. L’ambition du projet n’était pas de construire un simple agent conversationnel s’appuyant sur les connaissances générales d’un grand modèle de langage, mais de développer un **backend capable de retrouver des passages pertinents dans une base documentaire propre à l’administration**, puis de transmettre ces passages à un modèle de langage afin que celui-ci génère une réponse strictement contrainte par le contexte récupéré.

Cette approche répond à une exigence centrale du domaine administratif : la réponse fournie à l’utilisateur ne doit pas être une reformulation approximative issue des connaissances générales du modèle, mais une synthèse fidèle d’un contenu documentaire vérifiable.

1.2 Problématique métier et technique

1.2.1 Problématique métier

D’un point de vue métier, la problématique consiste à permettre à un utilisateur de poser une question en langage naturel sur des documents administratifs et d’obtenir une réponse exploitable, sans devoir parcourir manuellement l’ensemble des documents disponibles. Cette problématique recouvre des enjeux concrets :

- réduire le temps de recherche d’information pour l’usager ou l’agent administratif ;
- garantir que la réponse fournie reste fidèle au contenu réglementaire ou procédural existant ;
- permettre une mise à jour continue de la base de connaissances sans dépendre d’un cycle de développement long ;
- supporter une population d’utilisateurs s’exprimant en français comme en arabe, avec des formulations parfois éloignées du vocabulaire administratif formel.

1.2.2 Limites des grands modèles de langage classiques

Le problème technique traité par le projet découle d’une limite bien identifiée des grands modèles de langage (LLM) génératifs. Un LLM possède des connaissances générales acquises lors de son entraînement et peut produire une réponse fluide et grammaticalement correcte ; toutefois, il ne constitue pas à lui seul une base documentaire fiable pour un corpus administratif précis, évolutif et potentiellement multilingue. Plusieurs limitations concrètes ont motivé la conception d’une architecture RAG plutôt qu’un simple usage direct d’un LLM :

Hallucinations Un LLM interrogé sans contexte documentaire peut produire une réponse plausible en apparence, mais non fondée sur un texte réglementaire réel,

ce qui est inacceptable dans un contexte administratif où l'exactitude prime sur la fluidité.

Absence de mise à jour des connaissances Les connaissances paramétriques d'un LLM sont figées au moment de son entraînement. Or les procédures administratives, les formulaires et les références réglementaires évoluent, ce qui impose de pouvoir modifier la base de connaissances sans réentraîner le modèle.

Multilinguisme français/arabe Les questions posées par les usagers peuvent être formulées en français, en arabe, voire dans des registres différents, ce qui impose une capacité de retrieval robuste aux variations linguistiques.

Précision lexicale sur les noms propres et sigles Les documents administratifs contiennent de nombreux noms propres, sigles, références de formulaires et termes techniques exacts, pour lesquels une recherche purement sémantique par embeddings peut s'avérer insuffisante si elle n'est pas complétée par une recherche lexicale stricte.

1.2.3 Difficultés techniques associées

Au-delà de la problématique métier, la problématique technique du projet s'est avérée plus exigeante qu'une simple interface de chat connectée à un LLM. Pour produire des réponses suffisamment fiables, plusieurs difficultés devaient être traitées simultanément :

1. **Retrieval sémantique** : retrouver le bon passage documentaire même lorsque les termes employés par l'utilisateur ne correspondent pas exactement à ceux présents dans le document source.
2. **Recherche lexicale** : conserver une capacité de recherche fiable sur les noms propres, sigles, références et termes administratifs exacts, pour lesquels une approche purement sémantique est insuffisante.
3. **Multilinguisme** : prendre en compte notamment le français et l'arabe, avec la possibilité de questions formulées dans des registres ou des formulations différentes, y compris en darija dans certains cas observés durant le développement.
4. **Réduction des hallucinations** : forcer le modèle de génération à utiliser exclusivement le contexte fourni et à signaler explicitement lorsqu'une information ne peut pas être justifiée par ce contexte.
5. **Latence** : limiter le coût temporel cumulé des différentes étapes du pipeline (expansion de requête, retrieval, reranking, génération), ce coût pouvant devenir prohibitif si chaque composant est ajouté sans arbitrage.
6. **Maintenance de la base de connaissance** : permettre l'ajout, la suppression et la mise à jour de documents sans devoir reconstruire manuellement l'ensemble de l'application ni redémarrer le serveur pour chaque modification.

Ces six axes ont structuré l'ensemble des choix d'architecture présentés dans les chapitres suivants, chaque composant du pipeline final répondant à une ou plusieurs de ces difficultés.

1.3 Objectifs fonctionnels et techniques du système RAG

À partir des difficultés identifiées, un ensemble d'objectifs fonctionnels et techniques a été défini et progressivement raffiné au cours du développement. Le tableau 1.1 synthétise ces objectifs et la manière dont chacun a été concrètement réalisé dans le système final.

TABLE 1.1 – Objectifs du système RAG et réalisation associée dans le projet

Objectif	Réalisation dans le projet
Répondre à des questions sur le corpus documentaire	Pipeline RAG complet avec récupération de contexte puis génération par LLM
Améliorer le rappel (<i>recall</i>) du retrieval	Recherche hybride combinant BM25 et recherche dense
Réordonner les candidats retournés	Reranking par Cross-Encoder <code>BAAI/bge-reranker-base</code>
Gérer les formulations variées de l'utilisateur	Expansion de requête via un LLM léger dédié (Qwen)
Supporter plusieurs langues	Embeddings <code>intfloat/multilingual-e5-large</code> et consigne de réponse dans la langue de l'utilisateur
Maintenir la connaissance sans reconstruction manuelle	Endpoints CRUD d'upload, lecture, suppression et mise à jour
Réduire les appels coûteux répétés	Cache Redis avec durée de vie (TTL) de 24 heures
Exposer le pipeline à un frontend ou à un autre client	API REST développée avec FastAPI
Préparer l'exécution dans un environnement reproductible	Image Docker, fichier <code>requirements.txt</code> et variables d'environnement
Mesurer les performances du système	Temps d'expansion, de retrieval, de reranking, de génération LLM et nombre de tokens, retournés par l'endpoint <code>/ask</code>

Ces objectifs traduisent une volonté explicite de ne pas limiter le projet à un simple prototype de recherche documentaire, mais de construire un **backend opérationnel**, instrumenté et maintenable, capable d'évoluer après sa mise en production sans intervention manuelle lourde sur le code.

1.4 Périmètre fonctionnel du projet

1.4.1 Deux sous-systèmes complémentaires

Le périmètre final du projet couvre deux sous-systèmes complémentaires et interdépendants :

- **Knowledge Base / Document Management** : gestion des documents et de leurs fragments (*chunks*) au sein d'une base MongoDB Atlas, avec les opérations classiques de création, lecture, mise à jour et suppression ;
- **Question Answering** : interrogation du corpus documentaire via l'endpoint `/ask`, avec génération d'une réponse contextualisée à partir des passages les plus pertinents.

Le backend a été conçu comme un composant destiné à être consommé par une interface frontend externe. L'intégration frontend ne constitue toutefois pas la pièce centrale du dépôt final ; le travail de stage documenté dans ce rapport porte principalement sur le **backend RAG**, c'est-à-dire le retrieval, l'ingestion documentaire et les services associés (cache, API, conteneurisation).

1.4.2 Évolution du corpus au cours du stage

Le corpus et la méthode de préparation des données ont évolué de manière significative durant les expérimentations, reflétant une démarche itérative typique d'un projet de recherche appliquée.

Phase initiale : données de services publics numériques

Les premiers travaux du stage ont porté sur l'étude de données issues de services web publics et de structures de type catalogue de services, notamment des jeux de données de la forme **nos-services** et **nos-metiers**. Cette phase exploratoire avait pour objectif d'identifier les champs utiles, les textes descriptifs et les métadonnées susceptibles de servir de base à un système de recherche documentaire.

Recentrage sur le corpus administratif du stage

Le projet a ensuite été recentré sur un **corpus du stage**, principalement constitué de documents administratifs au format Word (`.docx`) contenant les informations nécessaires pour répondre aux questions des utilisateurs. Ce recentrage a constitué un tournant méthodologique important : il a orienté l'ensemble des choix d'ingénierie ultérieurs (extraction de texte, découpage en fragments, indexation) vers un format documentaire structuré et homogène, plus représentatif d'un usage administratif réel que les données de catalogue de services initialement étudiées.

Des fichiers PDF présents dans l'espace de travail ont également été mobilisés dans le processus de test et de questionnement du système, tandis que les réponses de référence utilisées pour l'évaluation restaient associées au corpus documentaire principal.

Généralisation vers un backend multi-formats

Le travail final s'est ensuite orienté vers une architecture de backend capable de traiter directement plusieurs formats de fichiers : `.docx`, `.pdf`, `.xls` et `.xlsx`. Cette généralisation, bien que l'usage métier principal étudié durant le stage soit resté le document administratif textuel au format Word, a permis de doter le système d'une capacité d'ingestion plus large, anticipant une diversité de sources documentaires que pourrait rencontrer une administration dans un contexte réel de déploiement.

Le schéma 1.1 synthétise cette trajectoire d'évolution du corpus, qui sera reprise et détaillée sur le plan technique dans le chapitre consacré à l'architecture et aux choix techniques.

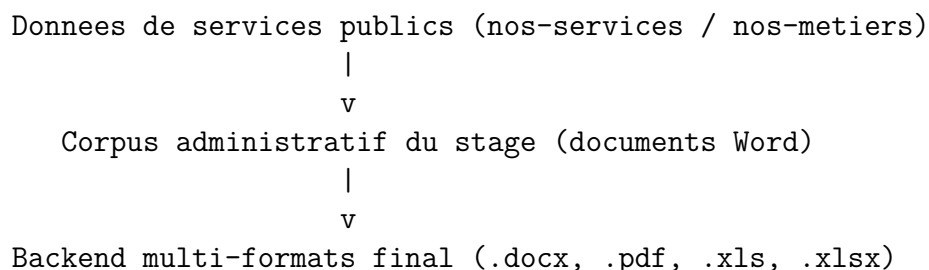


FIGURE 1.1 – Évolution du corpus documentaire au cours du stage

1.4.3 Synthèse du positionnement du projet

En résumé, le projet ne doit pas être appréhendé comme un simple chatbot s'appuyant sur un LLM générique, mais comme la construction progressive d'un **backend RAG complet et opérationnel**, répondant à une problématique métier concrète — l'accès simplifié et fiable à l'information administrative marocaine — au moyen d'une chaîne de composants techniques spécialisés : ingestion multi-formats, découpage en fragments, génération d'embeddings multilingues, recherche hybride, reranking, expansion de requête, génération contrainte par le contexte, mise en cache et exposition via une API REST conteneurisée. Les chapitres suivants détaillent successivement l'architecture retenue, les choix techniques opérés, les étapes de développement et les résultats expérimentaux obtenus.

Chapitre 2

État de l'Art et Fondements Théoriques du Retrieval-Augmented Generation

2.1 Introduction

Ce chapitre vise à poser les fondements théoriques nécessaires à la compréhension des choix d'architecture opérés durant le stage. Le système développé repose sur le paradigme du *Retrieval-Augmented Generation* (RAG), introduit initialement par Lewis et al. (2020), qui combine un mécanisme de récupération d'information (*retrieval*) à un modèle de génération de langage (*generation*). Nous détaillons successivement : (i) la justification théorique du RAG par rapport au *fine-tuning*, (ii) les fondements mathématiques de la recherche lexicale sparse (BM25), (iii) les principes de la recherche vectorielle dense et des espaces d'embeddings multilingues, (iv) l'architecture des mécanismes de reranking par Cross-Encoder, et (v) les principes de la *query expansion*. Chacune de ces briques théoriques correspond directement à un composant implémenté dans la version finale du système, décrite au chapitre suivant.

2.2 Le Paradigme RAG face au Fine-Tuning

2.2.1 Limites du LLM comme base de connaissance autonome

Un grand modèle de langage (LLM) encode ses connaissances de manière implicite dans les poids de son réseau de neurones, à l'issue d'un entraînement sur un corpus massif figé à une date donnée. Cette architecture pose trois limites majeures dans un contexte applicatif tel que celui des services publics numériques marocains :

1. **Obsolescence des connaissances** : les procédures administratives évoluent,

alors que les poids du modèle restent figés après l’entraînement ;

2. **Absence de traçabilité** : un LLM ne peut pas justifier explicitement la provenance d’une affirmation, car l’information n’est pas stockée sous une forme adressable ;
3. **Hallucination** : en l’absence de contexte contraignant, un LLM peut générer une réponse syntaxiquement plausible mais factuellement erronée, un phénomène particulièrement problématique dans un domaine administratif où l’exactitude prime sur la fluidité.

2.2.2 Deux approches concurrentes : Fine-Tuning vs RAG

Deux stratégies principales permettent d’adapter un LLM à un corpus spécifique :

Fine-tuning consiste à poursuivre l’entraînement du modèle sur un corpus spécialisé, modifiant directement ses paramètres θ afin de minimiser une fonction de perte sur le nouveau domaine :

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{domaine}}} [\mathcal{L}(f_{\theta}(x), y)] \quad (2.1)$$

Cette approche intègre la connaissance de manière paramétrique et diffuse dans le réseau, ce qui la rend coûteuse à mettre à jour et impossible à auditer précisément.

Retrieval-Augmented Generation conserve les paramètres θ du LLM figés et introduit un mécanisme de récupération R qui sélectionne, pour une requête q , un sous-ensemble de passages pertinents $C = \{c_1, \dots, c_k\}$ issus d’une base documentaire externe $\mathcal{D}$:

$$C = R(q, \mathcal{D}) = \underset{c \in \mathcal{D}}{\text{top-}k \text{ sim}}(q, c) \quad (2.2)$$

La génération finale est alors conditionnée non seulement par la requête, mais aussi par le contexte récupéré :

$$p(y \mid q) = p_{\theta}(y \mid q, C), \quad C = R(q, \mathcal{D}) \quad (2.3)$$

2.2.3 Justification du choix RAG pour le corpus administratif

Le choix du paradigme RAG pour ce projet repose sur trois arguments théoriques directement transposés dans l’implémentation finale (`v25/retrieval/rag.py`) :

- **Traçabilité** : la connaissance reste stockée sous forme de chunks adressables dans MongoDB Atlas, chacun associé à un identifiant MD5 et à un champ `source`, ce qui permet en principe de remonter à l’origine documentaire d’une réponse ;

- **Maintenabilité dynamique** : l'ajout, la suppression ou la mise à jour d'un document ne nécessitent pas de réentraînement du LLM, mais une simple opération CRUD suivie d'une resynchronisation des index (`sync_retrievers()`);
- **Réduction des hallucinations** : en contraignant le prompt système à n'utiliser que le contexte fourni (« *Answer ONLY from the provided context* »), le système transforme le LLM en générateur conditionné plutôt qu'en source autonome d'information, ce qui réduit — sans l'éliminer totalement — le risque de réponse non fondée.

Le compromis fondamental du RAG est ainsi déplacé : la qualité de la réponse finale ne dépend plus uniquement de la capacité générative du LLM, mais de manière critique de la **qualité du retrieval**. C'est cette dépendance qui justifie l'attention particulière portée, durant le stage, à la conception d'un mécanisme de recherche hybride sparse/dense couplé à un reranking fin, détaillés dans les sections suivantes.

2.3 Recherche Lexicale Sparse : le Modèle BM25

2.3.1 Principe général

BM25 (*Best Matching 25*) est une fonction de scoring probabiliste dérivée du modèle d'indépendance binaire (*Binary Independence Model*), largement utilisée en recherche d'information pour évaluer la pertinence d'un document d par rapport à une requête q sur la seule base de la fréquence des termes qu'ils partagent. Contrairement à une recherche dense, BM25 ne construit aucune représentation sémantique continue : elle opère directement sur les occurrences lexicales, ce qui la rend particulièrement adaptée à la récupération de termes exacts — noms de services publics, sigles administratifs, références réglementaires — pour lesquels une paraphrase sémantique serait risquée.

2.3.2 Formulation mathématique

Pour une requête q décomposée en termes $\{t_1, t_2, \dots, t_n\}$, le score BM25 d'un document d est défini par :

$$\text{score}(d, q) = \sum_{i=1}^n \text{IDF}(t_i) \cdot \frac{f(t_i, d) \cdot (k_1 + 1)}{f(t_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgl}}\right)} \quad (2.4)$$

où chaque composante possède une interprétation précise :

- $f(t_i, d)$: la fréquence brute du terme t_i dans le document d (*Term Frequency*, TF);
- $|d|$: la longueur du document d , mesurée en nombre de mots;

- avgdl : la longueur moyenne des documents dans la collection, servant de facteur de normalisation ;
- k_1 : un paramètre de saturation de la fréquence des termes, typiquement compris entre 1,2 et 2,0, qui contrôle à quelle vitesse le score cesse de croître lorsque $f(t_i, d)$ augmente (au-delà d’un certain seuil, répéter un terme n’apporte plus d’information supplémentaire) ;
- $b \in [0, 1]$: un paramètre de normalisation par la longueur du document, contrôlant dans quelle mesure les documents longs sont pénalisés (un document plus long a statistiquement plus de chances de contenir le terme par pur effet de volume) ;
- $\text{IDF}(t_i)$: l’*Inverse Document Frequency*, qui pondère l’importance du terme t_i en fonction de sa rareté dans l’ensemble de la collection $\mathcal{D}$, généralement défini par :

$$\text{IDF}(t_i) = \ln \left(\frac{N - n(t_i) + 0,5}{n(t_i) + 0,5} + 1 \right) \quad (2.5)$$

avec N le nombre total de documents (chunks) de la collection, et $n(t_i)$ le nombre de documents contenant le terme t_i . Un terme rare dans la collection (faible $n(t_i)$) reçoit un poids IDF élevé, car sa présence dans un document est fortement discriminante ; à l’inverse, un terme fréquent (par exemple un mot grammatical) reçoit un poids IDF proche de zéro.

2.3.3 Pertinence pour le corpus administratif marocain

Dans le contexte du projet, BM25 est instancié via la classe `BM25Retriever` de `LangChain`, construite dynamiquement à partir de l’ensemble des chunks présents dans la collection MongoDB (`BM25Retriever.from_documents(documents)`), avec un paramètre $k = 5$ correspondant au nombre de candidats retournés. Ce choix se justifie par le fait qu’un corpus administratif contient une forte proportion de **termes exacts non substituables** : noms d’organismes, intitulés de formulaires, références légales. Une recherche purement sémantique risquerait, dans ces cas, de privilégier un passage thématiquement proche mais lexicalement incorrect. BM25 agit ainsi comme un filet de sécurité lexical complémentaire à la recherche dense, décrite ci-après.

2.4 Recherche Vectorielle Dense : Similarité Sémantique dans un Espace d’Embeddings

2.4.1 Principe des embeddings sémantiques

La recherche dense repose sur le plongement (*embedding*) de chaque unité textuelle — requête ou chunk documentaire — dans un espace vectoriel continu $\mathbb{R}^d$, au moyen d’un modèle d’encodage neuronal $E : \text{texte} \rightarrow \mathbb{R}^d$. L’hypothèse sous-jacente est que la proximité géométrique dans cet espace reflète la proximité sémantique entre les textes, indépendamment de la formulation lexicale exacte employée :

$$\mathbf{v}_q = E(q), \quad \mathbf{v}_c = E(c), \quad c \in \mathcal{D} \quad (2.6)$$

Cette propriété permet de retrouver un passage pertinent même lorsque le vocabulaire de la question diffère de celui du document — une problématique centrale pour un corpus administratif où l’utilisateur final ne maîtrise pas nécessairement la terminologie officielle.

2.4.2 Similarité cosinus

La pertinence d’un chunk c par rapport à une requête q est mesurée par la similarité cosinus entre leurs représentations vectorielles :

$$\text{sim}_{\cos}(\mathbf{v}_q, \mathbf{v}_c) = \frac{\mathbf{v}_q \cdot \mathbf{v}_c}{\|\mathbf{v}_q\| \|\mathbf{v}_c\|} = \frac{\sum_{i=1}^d v_{q,i} v_{c,i}}{\sqrt{\sum_{i=1}^d v_{q,i}^2} \sqrt{\sum_{i=1}^d v_{c,i}^2}} \quad (2.7)$$

Lorsque les vecteurs sont préalablement normalisés ($\|\mathbf{v}\| = 1$), ce qui correspond exactement au paramétrage retenu dans l’implémentation finale (`encode_kwargs={"normalize_embeddings": True}`), la similarité cosinus se réduit à un simple produit scalaire :

$$\text{sim}_{\cos}(\mathbf{v}_q, \mathbf{v}_c) = \mathbf{v}_q \cdot \mathbf{v}_c \quad (2.8)$$

Cette simplification est déterminante en pratique, car elle permet à un moteur de recherche vectorielle tel que MongoDB Atlas Vector Search d’exécuter la recherche par produit scalaire, opération significativement moins coûteuse que le calcul explicite des normes à chaque comparaison.

2.4.3 Architecture Bi-Encoder et modèle multilingue retenu

La recherche dense repose sur une architecture dite **Bi-Encoder** : la requête et chaque document sont encodés *indépendamment* et *séparément*, sans interaction croisée au moment de l'encodage :

$$\mathbf{v}_q = E(q) \quad \text{et} \quad \mathbf{v}_c = E(c) \quad \text{calculés indépendamment} \quad (2.9)$$

Cette indépendance est ce qui rend la recherche dense *scalable* : les vecteurs $\mathbf{v}_c$ de tous les chunks de la collection peuvent être précalculés une seule fois et indexés, tandis que seule $\mathbf{v}_q$ doit être calculée à l'exécution, la comparaison avec l'ensemble de l'index se réduisant à une opération de recherche par plus proches voisins approximative (ANN).

Le modèle retenu dans la version finale du système est `intfloat/multilingual-e5-large`, encapsulé via `HuggingFaceEmbeddings` avec exécution sur CPU (`device: "cpu"`). Ce choix répond à deux contraintes du projet :

- **Couverture multilingue** : le modèle E5 multilingue projette des textes en français, arabe et autres langues dans un espace vectoriel partagé, ce qui permet théoriquement d'apparier une question posée dans une langue à un chunk rédigé dans une autre, sans traduction explicite préalable ;
- **Compromis qualité/infrastructure** : la variante *large* a été préférée à *small* au terme des expérimentations menées durant le stage (cf. Étape 4, chapitre 3), au prix d'un coût de calcul plus élevé mais jugé acceptable sur l'infrastructure CPU disponible.

Il convient de noter que la version de prototypage intermédiaire du projet reposait sur **ChromaDB** comme moteur de stockage et de recherche vectorielle. La version finale documentée dans ce rapport migre cette fonctionnalité vers **MongoDB Atlas Vector Search** (index nommé `vector_index`), afin de mutualiser au sein d'un même système la persistance des chunks, de leurs métadonnées, et de leur indexation vectorielle — ChromaDB ne constituant donc, dans l'architecture finale, qu'une étape historique d'expérimentation.

2.5 Fusion Hybride Sparse-Dense

Les deux mécanismes précédents captent des signaux de pertinence complémentaires mais non redondants : BM25 excelle sur l'exactitude lexicale, la recherche dense sur la proximité sémantique. Leur combinaison, opérée via un **EnsembleRetriever**, agrège les scores normalisés des deux retrievers selon une pondération linéaire :

$$\text{score}_{\text{hybride}}(c, q) = \alpha \cdot \widehat{\text{score}}_{\text{BM25}}(c, q) + (1 - \alpha) \cdot \widehat{\text{score}}_{\text{dense}}(c, q) \quad (2.10)$$

où $\widehat{\text{score}}$ désigne un score normalisé par rang (méthode usuelle des implémentations de type *Reciprocal Rank Fusion* sous-jacentes à LangChain), et $\alpha = 0,6$ dans la configuration finale retenue, traduisant une légère priorité accordée à la composante lexicale — un choix issu directement des expérimentations empiriques menées durant le stage plutôt que d’une justification théorique a priori.

2.6 Reranking par Cross-Encoder

2.6.1 Limite structurelle des Bi-Encoders

Le Bi-Encoder décrit en section 4.3 encode requête et document de manière totalement indépendante, ce qui interdit toute interaction fine entre leurs représentations avant le calcul de similarité. Cette contrainte architecturale, nécessaire à la scalabilité de la recherche initiale, limite mécaniquement la finesse du jugement de pertinence : deux passages peuvent obtenir un score cosinus proche alors que l’un répond précisément à la question et l’autre seulement thématiquement.

2.6.2 Architecture Cross-Encoder

Le **Cross-Encoder** lève cette limite en concaténant la requête et le passage candidat en une unique séquence d’entrée, traitée conjointement par un unique encodeur Transformer, permettant un mécanisme d’*attention croisée* entre tous les tokens de la question et tous les tokens du passage :

$$\text{score}_{\text{CE}}(q, c) = f_{\theta}([\text{CLS}] \ q \ [\text{SEP}] \ c \ [\text{SEP}]) \quad (2.11)$$

où f_{θ} désigne le Transformer encodeur suivi d’une couche de classification produisant un score scalaire de pertinence, typiquement calibré par une fonction sigmoïde :

$$\hat{y} = \sigma(W \cdot h_{[\text{CLS}]} + b) \quad (2.12)$$

Le tableau 2.1 synthétise la différence structurelle entre les deux architectures.

TABLE 2.1 – Comparaison Bi-Encoder vs Cross-Encoder

Critère	Bi-Encoder (retrieval)	Cross-Encoder (reranking)
Encodage	Requête et document séparés	Requête et document concaténés
Interaction	Aucune (produit scalaire a posteriori)	Attention croisée complète
Complexité	$O(1)$ par comparaison (index précalculé)	$O(n)$ ré-encodage à chaque requête
Scalabilité	Élevée (millions de documents)	Faible (dizaines de candidats)
Précision	Modérée	Élevée

Le prix de cette précision accrue est une complexité computationnelle bien supérieure : le Cross-Encoder ne peut pas précalculer de représentation indépendante pour chaque document, chaque paire (q, c) devant être ré-encodée intégralement à l’exécution. Cette contrainte interdit son usage direct sur l’ensemble d’une collection documentaire, d’où la nécessité d’une architecture en deux étages.

2.6.3 Architecture en deux étages : rappel puis précision

Le compromis retenu dans le système consiste à appliquer le Cross-Encoder uniquement sur l’ensemble restreint des candidats déjà filtrés par la recherche hybride sparse/dense, selon le schéma classique de la littérature en recherche d’information (*retrieve-then-rerank*) :

$$C_{\text{final}} = \underset{c \in C_{\text{hybride}}}{\text{top-3}} \text{ score}_{\text{CE}}(q, c), \quad C_{\text{hybride}} = R_{\text{hybride}}(q, \mathcal{D}) \quad (2.13)$$

Le modèle retenu, **BAAI/bge-reranker-base**, est appliqué sur les paires (**question**, **chunk.page_content**) issues du retrieval hybride, et seuls les trois candidats au score le plus élevé sont transmis au LLM de génération. Ce choix architectural répond directement à la problématique identifiée en section 4.4 du document technique : un retrieval à fort rappel ne garantit pas que les meilleurs passages soient positionnés en tête de liste, fonction que le reranking vient précisément corriger.

2.7 Query Expansion : Augmentation du Rappel par Reformulation

2.7.1 Motivation théorique

La performance d’un système de retrieval dépend fortement de l’adéquation lexicale et sémantique entre la formulation de la requête utilisateur et celle du corpus documentaire. Or, un même besoin d’information peut s’exprimer sous des formulations très diverses — registre de langue, choix lexical, niveau de précision — en particulier dans un contexte multilingue où la question peut être posée en français, en arabe standard, voire en dialecte marocain (Darija). La *query expansion* vise à pallier ce désalignement lexical en générant, à partir de la question originale q , un ensemble de reformulations alternatives $\{q'_1, q'_2, \dots\}$ censées élargir la couverture de la recherche :

$$Q_{\text{expansion}} = \text{LLM}_{\text{exp}}(q) = \{q'_1, q'_2\} \quad (2.14)$$

L’ensemble des requêtes (originale et reformulées) est alors utilisé conjointement pour interroger les retrievers, avec pour objectif théorique d’augmenter le rappel (*recall*)

global du système :

$$\text{Recall}(Q_{\text{expansion}}) \geq \text{Recall}(\{q\}) \quad (2.15)$$

cette inégalité n’étant vérifiée en pratique que si les reformulations générées restent fidèles à l’intention initiale de l’utilisateur.

2.7.2 Implémentation retenue

Dans le système développé, cette expansion est déléguée à un modèle de langage dédié et distinct du modèle de génération finale, à savoir `qwen3.6-27b` servi via l’API Groq, avec une température fixée à 0 afin de maximiser la déterminisme des reformulations. Le prompt d’expansion, rédigé en arabe, impose des contraintes strictes de format : exactement deux requêtes alternatives, sans numérotation, sans préambule ni explication, employant un vocabulaire administratif formel.

2.7.3 Compromis rappel/latence/coût

La query expansion introduit néanmoins un coût non négligeable, illustré empiriquement lors des expérimentations menées durant le stage (détaillées au chapitre 5) : l’ajout de cette étape a été associé à une hausse sensible du nombre total de tokens consommés et du temps de traitement global, sans garantie systématique de gain qualitatif. Ce constat rejoint une problématique documentée dans la littérature sur la query expansion neuronale : une reformulation trop éloignée de la requête originale — en particulier pour des formulations en Darija, dont la normalisation vers un registre administratif formel peut altérer le sens initial — risque de dégrader la précision du retrieval plutôt que de l’améliorer. Ce constat justifie, comme piste d’amélioration identifiée en fin de stage, le passage d’une expansion systématique à une expansion **conditionnelle**, déclenchée uniquement lorsque la question est jugée ambiguë ou lexicalement éloignée du corpus.

2.8 Synthèse

Les fondements théoriques exposés dans ce chapitre définissent une chaîne de traitement cohérente où chaque composant compense les limites structurelles du précédent : le RAG substitue à la mémorisation paramétrique un mécanisme de récupération traçable et maintenable ; BM25 garantit l’exactitude lexicale que la recherche dense ne peut assurer seule ; la recherche dense, portée par les embeddings multilingues `multilingual-e5-large`, capture la proximité sémantique au-delà de la simple co-occurrence de termes ; le Cross-Encoder affine, au prix d’un coût computationnel ciblé sur un nombre restreint de candidats, un classement que l’architecture Bi-Encoder ne peut produire avec la

même finesse ; enfin, la query expansion tente d’élargir la couverture du retrieval au prix d’un arbitrage explicite entre rappel, latence et coût en tokens. Le chapitre suivant détaille la traduction concrète de ces principes théoriques dans l’architecture logicielle effectivement implémentée et livrée à l’issue du stage.

Chapitre 3

Architecture Technique et Conception du Système RAG

3.1 Introduction

Ce chapitre présente en détail l'architecture technique retenue pour la version finale du système, référencée `v25/retrieval/rag.py` dans le dépôt du projet. Contrairement à une présentation purement chronologique, l'objectif est ici de décrire l'état final du pipeline tel qu'il a été livré, en explicitant les choix de conception, les technologies mobilisées à chaque étage et les mécanismes de synchronisation qui garantissent la cohérence du système entre la base documentaire et les index de recherche.

Il convient de noter dès à présent que l'architecture finale repose sur un fichier unique et monolithique (`rag.py`), regroupant l'ensemble des composants applicatifs — configuration, connexions aux services externes, définition des retrievers, endpoints FastAPI et logique métier. Cette organisation, bien que non modulaire, correspond à la réalité du livrable de stage et sera assumée comme telle dans ce rapport, par opposition à une architecture en couches (`app/main.py`, `app/rag.py`, `app/retrieval.py`, etc.) qui n'a qu'une valeur indicative dans la documentation du projet.

3.2 Pipeline Séquentiel End-to-End

3.2.1 Vue d'ensemble du flux de traitement

Le système repose sur un pipeline séquentiel et linéaire, déclenché à chaque requête sur l'endpoint `/ask`. Ce pipeline peut être décomposé en deux grandes phases : une phase de vérification du cache, suivie, en cas d'absence de résultat en cache, d'une chaîne complète de traitement comprenant l'expansion de requête, la recherche hybride, le reranking et la génération de réponse par un grand modèle de langage.

Ce flux illustre un compromis architectural assumé : la robustesse du retrieval (grâce à l'hybridation et au reranking) est privilégiée au détriment d'une latence plus faible, cette dernière étant partiellement compensée par le mécanisme de cache.

3.3 Stack Technologique Final

Le tableau 3.1 récapitule l'ensemble des technologies retenues dans la version de référence v25, couche par couche.

Cette stack traduit un choix délibéré de mutualiser autant que possible les responsabilités de persistance et de recherche dense au sein d'un unique système, MongoDB Atlas, plutôt que de multiplier les services externes. Ce choix sera justifié plus en détail dans la section suivante.

3.4 Évolution du Vector Store : de ChromaDB à MongoDB Atlas Vector Search

3.4.1 Contexte historique

Au cours des phases intermédiaires de développement, la base vectorielle utilisée pour le stockage et la recherche dense était **ChromaDB**. Cette solution, légère et simple à mettre en œuvre localement, a permis de mener plusieurs expérimentations combinant différents modèles d'embeddings, différents rerankers et différentes stratégies de fusion sparse/dense. ChromaDB a ainsi joué un rôle de *sandbox* de prototypage rapide durant les premières itérations du projet (notamment autour des versions historiques antérieures à v25).

3.4.2 Transition vers MongoDB Atlas Vector Search

La version finale v25, telle qu'implémentée dans `rag.py`, remplace ChromaDB par **MongoDB Atlas Vector Search**, exploité via la classe `MongoDBAtlasVectorSearch` de LangChain et un index nommé `vector_index`. Ce changement d'architecture répond à un objectif de consolidation : disposer, au sein d'un unique système de base de données, à la fois de la persistance des chunks, de leurs métadonnées, et de leur indexation vectorielle, sans nécessiter de synchronisation entre deux systèmes de stockage distincts.

L'extrait ci-dessous illustre l'initialisation du vector store dans la version finale :

```
1 embeddings = HuggingFaceEmbeddings(  
2     model_name="intfloat/multilingual-e5-large",  
3     model_kwargs={"device": "cpu"},  
4     encode_kwargs={"normalize_embeddings": True})
```

TABLE 3.1 – Stack technologique de la version finale v25

Couche	Technologie / Choix	Rôle
Langage	Python	Implémentation du backend et du pipeline RAG
API	FastAPI	Exposition REST du chatbot et de la gestion documentaire
Validation API	Pydantic (QuestionRequest)	Validation et typage des requêtes entrantes sur /ask
Orchestration RAG	LangChain	Interfaces Document, retrievers, embeddings, vector store, LLM
Embeddings	intfloat/multilingual-e5-large	Vectorisation multilingue des chunks et des requêtes
Recherche dense	MongoDB Atlas Vector Search (MongoDBAtlasVectorSearch)	Recherche par similarité vectorielle
Recherche sparse	BM25Retriever	Recherche lexicale par termes exacts
Fusion hybride	EnsembleRetriever	Combinaison pondérée BM25 / dense (0.6 / 0.4)
Reranking	BAAI/bge-reranker-base (Cross-Encoder)	Réordonnancement fin des candidats
LLM Expansion	Qwen 3.6 27B (via Groq)	Génération de deux reformulations de la question
LLM Réponse	GPT-OSS 120B (via Groq)	Génération finale contrainte par le contexte
SDK LLM	langchain-groq	Intégration des modèles servis par Groq
Base documentaire	MongoDB Atlas	Stockage des chunks, métadonnées et recherche vectorielle
Cache	Redis / Upstash Redis	Cache exact des réponses avec expiration (TTL)
Traitement DOCX	python-docx	Extraction du texte des fichiers Word
Traitement PDF	PyPDF2	Extraction de texte page par page
Traitement Excel	pandas + openpyxl	Lecture des feuilles et conversion en texte
Configuration	python-dotenv	Chargement des variables d’environnement
TLS MongoDB	certifi	Fourniture du bundle d’autorités de certification
Conteneurisation	Docker	Image reproductible du backend
Serveur applicatif	Uvicorn	Exécution ASGI de l’application FastAPI

```
7 vector_store = MongoDBAtlasVectorSearch(  
8     collection=collection,  
9     embedding=embeddings,  
10    index_name="vector_index"  
11 )  
12  
13 dense = vector_store.as_retriever(search_kwargs={"k": 5})
```

Listing 3.1 – Initialisation du vector store MongoDB Atlas

3.4.3 Remarque sur la cohérence documentaire

Il est important de souligner, pour la rigueur du présent rapport, que le **README** du dépôt mentionne encore ChromaDB dans sa description architecturale et dans son tableau de technologies, alors que le code de production `v25/retrieval/rag.py` utilise effectivement MongoDB Atlas Vector Search. Cette divergence constitue une dette documentaire identifiée au cours du stage (voir chapitre 6.4, limite 6.2.2). Le présent rapport retient donc **MongoDB Atlas Vector Search comme architecture de production finale**, ChromaDB étant présenté exclusivement comme une étape de prototypage historique, désormais obsolète dans la version livrée.

3.4.4 Justification du choix final

Le passage à MongoDB Atlas Vector Search se justifie par plusieurs considérations :

- **Unification du stockage** : les chunks, leurs métadonnées (`source`, `id`) et leur représentation vectorielle résident dans la même collection `rag_db.chunks`, simplifiant les opérations CRUD.
- **Cohérence transactionnelle** : toute modification de la collection (ajout, suppression, mise à jour) est immédiatement reflétée dans le retriever dense, qui interroge directement le vector store.
- **Réduction de la complexité opérationnelle** : l'usage d'un service managé cloud (Atlas) limite la maintenance d'infrastructure comparée à une instance ChromaDB autogérée.

Ce choix a néanmoins une contrepartie, développée au chapitre 4 : contrairement au retriever dense, le retriever sparse (BM25) reste une structure construite en mémoire à partir d'un chargement complet de la collection, ce qui impose un mécanisme explicite de resynchronisation après toute opération d'écriture (fonction `sync_retrievers()`).

3.5 Stratégie de Caching Exact avec Redis/Upstash

3.5.1 Principe général

Le système intègre un mécanisme de cache applicatif fondé sur Redis (hébergé via le service cloud Upstash), dont l'objectif est d'éviter de réexécuter l'intégralité de la chaîne RAG — expansion, retrieval, reranking, génération — lorsqu'une question strictement identique a déjà été traitée.

Il s'agit explicitement d'un **cache exact**, fondé sur une correspondance stricte de la chaîne de caractères normalisée, et non d'un *semantic cache* capable de rapprocher des formulations sémantiquement équivalentes mais lexicalement différentes. Cette distinction est fondamentale : deux questions telles que « Comment obtenir un extrait d'acte de naissance ? » et « Quelle est la procédure pour un extrait de naissance ? » ne partageront pas la même entrée de cache, malgré leur équivalence sémantique.

3.5.2 Normalisation et génération de la clé

La question est d'abord normalisée par suppression des espaces superflus et mise en minuscules, puis hachée en MD5 pour constituer la clé de cache :

```
1 normalized_question = request.question.strip().lower()
2 cache_key = (
3     f"rag_cache:{hashlib.md5(normalized_question.encode('utf-8')).hexdigest()}"
4 )
```

Listing 3.2 – Normalisation et construction de la clé de cache

Le préfixe `rag_cache:` permet d'isoler l'espace de noms des clés relatives au pipeline RAG au sein de l'instance Redis partagée, facilitant d'éventuelles opérations de purge ciblée.

3.5.3 Lecture et écriture du cache

La lecture du cache est effectuée en tout début de traitement de la requête, avant même l'étape d'expansion :

```
1 cached_response = redis_client.get(cache_key)
2 if cached_response:
3     response_data = json.loads(cached_response)
4     response_data["cached"] = True
5     return response_data
```

Listing 3.3 – Lecture du cache avant exécution du pipeline

En cas d'absence d'entrée (*cache miss*), la réponse finale, une fois calculée, est stockée avec une durée de vie fixe de 24 heures :

```
1 redis_client.setex(  
2     cache_key,  
3     86400,  
4     json.dumps(final_response)  
5 )
```

Listing 3.4 – Écriture du cache avec TTL de 24 heures

Le choix d'une durée de vie de 86400 secondes, soit 24 heures, découle d'un compromis entre fraîcheur de l'information et taux de réutilisation du cache : cette durée est suffisamment longue pour absorber une charge répétitive sur une même journée d'utilisation, tout en évitant qu'une réponse obsolète ne persiste trop longtemps après une éventuelle modification du corpus documentaire.

3.5.4 Invalidation du cache

Afin de garantir la cohérence entre les réponses servies et l'état courant de la base documentaire, toute opération de modification du corpus (ajout, suppression, mise à jour d'un document) déclenche un appel à une fonction `clear_cache()`, qui vide intégralement le cache. Cette stratégie, volontairement simple, privilégie la cohérence des réponses au détriment d'une invalidation plus fine (par exemple ciblée sur les seules questions liées au document modifié), qui aurait nécessité un système d'indexation supplémentaire entre questions en cache et documents sources.

3.5.5 Migration de Redis local vers Redis cloud

Il convient de noter que l'usage de Redis en instance locale (`localhost:6379`) s'est heurté, en cours de développement, à un problème de disponibilité du service dans l'environnement de test. Cette contrainte a conduit à migrer vers une instance Redis managée dans le cloud, via le service Upstash, configuré par la variable d'environnement `UPSTASH_REDIS_URL` :

```
1 redis_url = os.getenv("UPSTASH_REDIS_URL")  
2 redis_client = redis.from_url(  
3     redis_url,  
4     decode_responses=True,  
5     socket_connect_timeout=2,  
6     socket_timeout=2  
7 )
```

Listing 3.5 – Connexion au client Redis via Upstash

Les paramètres `socket_connect_timeout` et `socket_timeout`, fixés à deux secondes, visent à éviter qu'une indisponibilité temporaire du service de cache ne bloque durablement le traitement d'une requête utilisateur.

3.6 API REST FastAPI

3.6.1 Principes de conception de l'API

L'ensemble du pipeline RAG est exposé au travers d'une API REST développée avec FastAPI, choisie pour sa performance asynchrone, sa validation native des schémas de données via Pydantic, et sa capacité à générer automatiquement une documentation interactive (OpenAPI/Swagger). Cette couche API a pour rôle de découpler la logique de traitement RAG de toute interface utilisateur ou client final, permettant une consommation par un frontend web ou tout autre système tiers.

3.6.2 Validation des entrées avec Pydantic

La requête adressée à l'endpoint `/ask` est validée au moyen d'un modèle Pydantic dédié, garantissant que le corps de la requête contient bien un champ `question` de type chaîne de caractères avant toute exécution du pipeline. Cette validation en amont permet d'éviter la propagation d'entrées malformées dans les étapes coûteuses du pipeline (expansion, retrieval, génération).

3.6.3 Cartographie des endpoints

Le tableau 3.2 présente l'ensemble des routes exposées par le backend dans sa version finale.

3.6.4 Exemple d'appel à l'endpoint principal

L'endpoint central `/ask` reçoit une question en langage naturel et retourne une réponse structurée incluant la réponse elle-même, ainsi que des métriques d'observabilité détaillées :

```
1 POST /ask
2 Content-Type: application/json
3
4 {
5   "question": "Comment obtenir un document administratif ?"
6 }
```

Listing 3.6 – Exemple de requête sur l'endpoint `/ask`

TABLE 3.2 – Endpoints REST exposés par le backend v25

Méthode	Route	Rôle
GET	/	Vérification de disponibilité et de version du service (<i>health check</i>)
GET	/documents	Liste de l'ensemble des chunks/documents présents en base
GET	/document/{filename}	Récupération des chunks associés à un fichier donné
DELETE	/documents/{filename}	Suppression de tous les chunks associés à un fichier
PUT	/document/{filename}	Remplacement et re-ingestion complète d'un document
POST	/upload_file	Ajout d'un nouveau document au corpus
POST	/ask	Question-réponse RAG (point d'entrée principal)

```

1 {
2   "Question": "...",
3   "Answer": "...",
4   "total_time": 2.1,
5   "expansion_time": 0.4,
6   "retrieval_time": 0.3,
7   "reranker_time": 0.5,
8   "llm_time": 0.9,
9   "token_usage": {
10    "total_tokens": 123,
11    "prompt_tokens": 100,
12    "completion_tokens": 23
13  },
14   "cached": false
15 }

```

Listing 3.7 – Exemple de structure de réponse de l'endpoint /ask

Il est à noter que les valeurs numériques présentées ci-dessus constituent un exemple de structure de réponse ; les valeurs réelles de latence et de consommation de tokens sont mesurées dynamiquement à chaque exécution, comme détaillé au chapitre 4.

3.6.5 Gestion documentaire via CRUD

Les opérations de gestion documentaire (POST /upload_file, DELETE /documents/{filename}, PUT /document/{filename}) suivent systématiquement le même schéma de traitement : extraction et chunking du contenu, écriture ou suppression

dans la collection MongoDB, puis **synchronisation obligatoire des retrievers** (fonction `sync_retrievers()`) et **invalidation du cache** (fonction `clear_cache()`). Ce couplage systématique entre modification documentaire et resynchronisation des structures de recherche constitue l'un des points architecturaux les plus significatifs du système, et fait l'objet d'une analyse détaillée au chapitre suivant consacré aux défis techniques rencontrés.

3.7 Synthèse du chapitre

L'architecture décrite dans ce chapitre traduit une démarche de conception pragmatique, dans laquelle chaque composant répond à une problématique technique précise : la recherche hybride BM25/dense pour couvrir à la fois l'exactitude lexicale et la similarité sémantique, le reranking par Cross-Encoder pour affiner la sélection finale des chunks, le cache Redis pour limiter les appels redondants au pipeline complet, et une API FastAPI pour exposer ces capacités de manière normalisée. La consolidation du stockage documentaire et de l'indexation vectorielle au sein de MongoDB Atlas, en remplacement de la solution ChromaDB utilisée en phase de prototypage, constitue la décision architecturale structurante de la version finale v25. Le chapitre suivant détaille les défis techniques rencontrés lors de la mise en œuvre de cette architecture ainsi que les solutions apportées.

Chapitre 4

Implémentation, Pipeline d’Ingestion et Synchronisation d’État

4.1 Introduction

Ce chapitre détaille les aspects algorithmiques et logiciels de la version finale du système, telle qu’implémentée dans le fichier monolithique `v25/retrieval/rag.py`. Contrairement à une architecture modulaire idéalisée (de type `app/main.py`, `app/ingestion.py`, `app/retrieval.py`), la version livrée concentre l’ensemble de la logique métier — configuration, ingestion multi-format, retrieval hybride, reranking, génération et gestion du cache — au sein d’un unique fichier. Ce choix d’implémentation, bien qu’il limite la modularité, a permis d’itérer rapidement sur les différents composants du pipeline durant le stage.

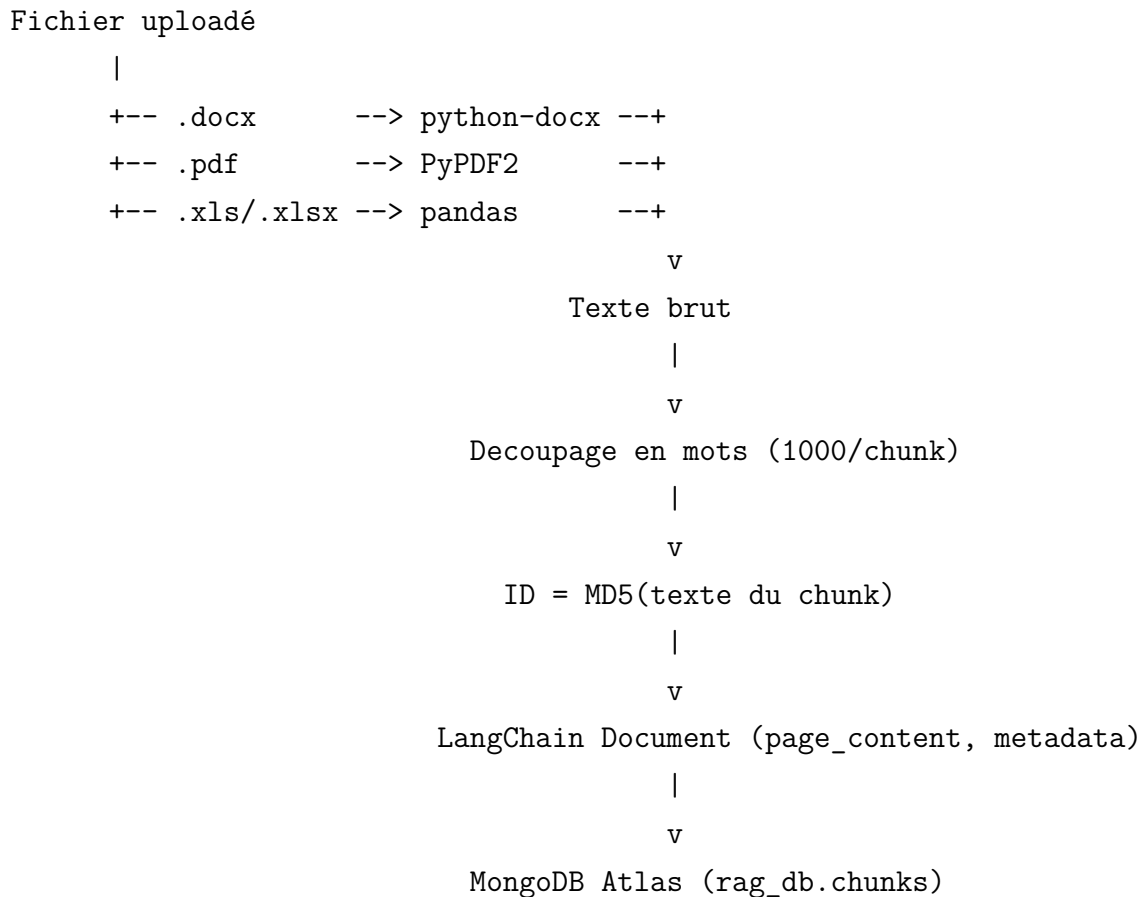
L’objectif de ce chapitre est double : premièrement, présenter en détail le pipeline d’ingestion documentaire multi-format (DOCX, PDF, XLS/XLSX) et la stratégie de chunking retenue ; deuxièmement, analyser un problème architectural central rencontré durant le développement — la désynchronisation entre l’état persistant (MongoDB Atlas) et l’état en mémoire (index BM25) — ainsi que la solution logicielle apportée. Enfin, la stratégie de conteneurisation Docker du backend est présentée et discutée.

4.2 Pipeline d’Ingestion Multi-Format

4.2.1 Vue d’ensemble

Le système accepte quatre formats de fichiers en entrée du point de terminaison `POST /upload_file` : `.docx`, `.pdf`, `.xls` et `.xlsx`. Chaque format dispose d’une fonction

d'extraction dédiée, mais tous convergent vers une même logique de découpage en chunks, illustrée par la figure conceptuelle suivante :



Ce choix d'une fonction de découpage *commune* en aval de trois extracteurs *spécifiques* permet de garantir une homogénéité du format de stockage, quel que soit le type de document source, tout en isolant la complexité propre à chaque format dans une étape d'extraction dédiée.

4.2.2 Extraction des documents Word (.docx)

L'extraction des fichiers Word repose sur la bibliothèque `python-docx`, qui expose la structure interne du document sous forme d'une collection de paragraphes. Le parseur parcourt séquentiellement ces paragraphes et concatène leur contenu textuel, en insérant un saut de ligne entre chaque paragraphe afin de préserver, dans une certaine mesure, la structure logique du document d'origine.

```

1 def parse_docx(content: bytes):
2     try:
3         doc = DocxDocument(io.BytesIO(content))
4         text = ""
5         for paragraph in doc.paragraphs:
6             text += paragraph.text + "\n"
7 
```

```
8     chunks = []
9     chunk_size = 1000
10    words = text.split()
11    for i in range(0, len(words), chunk_size):
12        chunk_text = " ".join(words[i:i+chunk_size])
13        if chunk_text.strip():
14            chunks.append({
15                "text": chunk_text,
16                "id": hashlib.md5(chunk_text.encode()).
17               hexdigest()
18            })
19    return chunks
20 except Exception as e:
21     print(f"Error {e} while loading docx")
22     return []
```

Listing 4.1 – Extraction du texte d'un fichier DOCX

On notera la présence d'un bloc `try/except` englobant, qui garantit qu'une erreur de parsing sur un document corrompu ou mal formé n'interrompt pas le service : la fonction retourne une liste vide de chunks plutôt que de propager une exception non gérée jusqu'au client de l'API. Ce choix privilégie la disponibilité du service à la complétude stricte de l'ingestion, ce qui constitue un compromis raisonnable pour un backend destiné à recevoir des documents administratifs de provenance potentiellement hétérogène.

4.2.3 Extraction des documents PDF (.pdf)

L'extraction PDF s'appuie sur PyPDF2 et procède page par page, chaque page étant traitée indépendamment via la méthode `extract_text()`. Le texte extrait de chaque page est concaténé au texte global du document.

```
1 def parse_pdf(content: bytes):
2     pdf_reader = PdfReader(io.BytesIO(content))
3     text = ""
4     for page in pdf_reader.pages:
5         text += (page.extract_text() or "") + "\n"
6
7     chunks = []
8     chunk_size = 1000
9     words = text.split()
10    for i in range(0, len(words), chunk_size):
11        chunk_text = " ".join(words[i:i+chunk_size])
12        if chunk_text.strip():
13            chunks.append({
14                "text": chunk_text,
```

```
15         "id": hashlib.md5(chunk_text.encode()).hexdigest()
16     })
17     return chunks
```

Listing 4.2 – Extraction du texte d’un fichier PDF

L’expression `page.extract_text() or ""` constitue une garde défensive essentielle : certaines pages, notamment celles composées principalement d’images scannées sans couche de texte, retournent une valeur `None` plutôt qu’une chaîne vide. L’opérateur logique `or` substitue alors une chaîne vide, évitant une erreur de concaténation de type `TypeError`. Il convient cependant de souligner une limite importante à ce stade : `PyPDF2` n’effectue *aucune* reconnaissance optique de caractères (OCR). Un document PDF scanné sans couche texte produira donc un ensemble de chunks vides, silencieusement ignorés par la condition `if chunk_text.strip()`.

4.2.4 Extraction des fichiers Excel (.xls / .xlsx)

L’extraction Excel diffère structurellement des deux précédentes, dans la mesure où la donnée source est tabulaire et non un flux de texte continu. Le parseur s’appuie sur `pandas` (avec le moteur `openpyxl` pour la lecture des fichiers `.xlsx`) et convertit chaque colonne de chaque feuille en une représentation textuelle linéarisée.

```
1 def parse_excel(content: bytes):
2     excel_file = pd.ExcelFile(io.BytesIO(content))
3     all_text = ""
4     for sheet_name in excel_file.sheet_names:
5         df = pd.read_excel(excel_file, sheet_name=sheet_name)
6         for col in df.columns:
7             values = df[col].dropna().astype(str).to_list()
8             if values:
9                 all_text += f"Column {col}: " + ", ".join(values) +
10                 "\n"
11
12     chunks = []
13     chunk_size = 1000
14     words = all_text.split()
15     for i in range(0, len(words), chunk_size):
16         chunk_text = " ".join(words[i:i+chunk_size])
17         if chunk_text.strip():
18             chunks.append({
19                 "text": chunk_text,
20                 "id": hashlib.md5(chunk_text.encode()).hexdigest()
21             })
22     return chunks
```

Listing 4.3 – Extraction et linéarisation d’un fichier Excel

Cette stratégie de linéarisation *par colonne* (plutôt que par ligne) implique un choix de représentation notable : la sémantique de chaque enregistrement tabulaire (une ligne du tableau, correspondant typiquement à une entité ou un enregistrement administratif) est diluée au profit d'une agrégation verticale des valeurs. La méthode `dropna()` élimine les cellules vides avant conversion en chaîne, ce qui évite l'insertion de valeurs "nan" dans le texte final. Cette approche, bien que simple à implémenter, constitue une limite identifiée du système (cf. section 6.2.5) : elle est adaptée à des feuilles de type catalogue ou nomenclature, mais moins pertinente pour des tableaux à forte structure relationnelle entre colonnes.

4.2.5 Stratégie de chunking : découpage fixe sans chevauchement

Description du mécanisme

Un point de vigilance central pour la fidélité du rapport concerne la stratégie de chunking effectivement utilisée dans la version finale. Alors que des étapes intermédiaires du projet ont exploré l'utilisation du `RecursiveCharacterTextSplitter` de `LangChain` (avec des paramètres `chunk_size=1000` et `chunk_overlap=150`), le code de `v25/retrieval/rag.py` implémente une logique de découpage **directe, fondée sur le comptage de mots, sans chevauchement**.

Formellement, pour un texte source T décomposé en une séquence de mots $w_1, w_2, \dots, w_n$ et une taille de chunk fixée à $c = 1000$, le k -ième chunk C_k est défini par :

$$C_k = \{w_i \mid i \in [k \cdot c, \min((k+1) \cdot c, n))\}, \quad k = 0, 1, \dots, \left\lceil \frac{n}{c} \right\rceil - 1 \quad (4.1)$$

Cette définition traduit directement la boucle Python `for i in range(0, len(words), chunk_size)`, où chaque chunk regroupe exactement 1000 mots consécutifs (à l'exception du dernier chunk, potentiellement plus court). L'absence de chevauchement (`overlap = 0`) signifie qu'un mot appartenant au chunk C_k n'apparaît jamais dans le chunk C_{k+1} .

Génération de l'identifiant par hachage MD5

Chaque chunk se voit attribuer un identifiant déterministe, calculé par hachage MD5 du texte du chunk lui-même :

$$\text{id}(C_k) = \text{MD5}(\text{encode}(C_k)) \quad (4.2)$$

```
1 chunk_text = " ".join(words[i:i+chunk_size])
```

```
2 if chunk_text.strip():
3     chunks.append({
4         "text": chunk_text,
5         "id": hashlib.md5(chunk_text.encode()).hexdigest()
6     })
```

Listing 4.4 – Génération de l'identifiant de chunk par hachage MD5

Ce choix présente un double avantage. D'une part, il fournit un identifiant unique et reproductible sans dépendre d'un compteur auto-incrémenté ou d'un UUID généré aléatoirement, ce qui simplifie la traçabilité d'un chunk entre plusieurs exécutions du pipeline d'ingestion. D'autre part, il confère au système une forme de *déduplication implicite* : deux chunks strictement identiques en contenu textuel — par exemple si le même document est ingéré deux fois par erreur — produiront le même hash MD5, ce qui peut être exploité pour détecter des doublons lors d'opérations de maintenance ultérieures.

Compromis architectural

Le choix d'un découpage fixe sans chevauchement, plutôt qu'un splitter récursif avec chevauchement, doit être analysé comme un arbitrage entre simplicité d'implémentation et qualité potentielle du retrieval. L'absence d'overlap introduit un risque identifié : une information pertinente située à cheval sur la frontière entre deux chunks — par exemple une phrase coupée entre les mots 1000 et 1001 — peut être fragmentée de telle sorte qu'aucun des deux chunks ne contient, isolément, un contexte suffisant pour répondre correctement à une question portant sur cette information. Ce compromis est documenté comme une limite connue du système (section 6.2.3), et constitue l'un des axes d'amélioration identifiés en fin de stage.

4.2.6 Construction des objets Document LangChain

Une fois les chunks produits par l'un des trois extracteurs, ceux-ci sont convertis en objets `Document` de LangChain, structure standardisée qui associe le contenu textuel à un dictionnaire de métadonnées :

```
1 docs_to_add = [
2     Document(
3         page_content=chunk["text"],
4         metadata={"source": file.filename, "id": chunk["id"]}
5     )
6     for chunk in chunks
7 ]
```

Listing 4.5 – Construction des objets Document avec métadonnées

Le champ **source**, correspondant au nom du fichier d’origine, joue un rôle structurant dans l’architecture du système : il constitue la clé utilisée par les opérations de lecture, de suppression et de mise à jour documentaire (cf. section 4.4.2), permettant de regrouper l’ensemble des chunks issus d’un même fichier sans nécessiter de table de jointure additionnelle.

4.3 Le Problème de Désynchronisation d’État

4.3.1 Origine du problème

L’architecture de retrieval hybride du système repose sur deux mécanismes de nature fondamentalement différente :

- le **retrieveur dense**, construit comme une vue directe sur MongoDB Atlas Vector Search via `MongoDBAtlasVectorSearch.as_retriever()` : chaque requête interroge la base en temps réel, et reflète donc *instantanément* toute modification de la collection `rag_db.chunks` ;
- le **retrieveur BM25**, construit via `BM25Retriever.from_documents()` : cette structure est calculée *une fois*, en mémoire, à partir d’une liste de documents chargée au moment de l’initialisation, et n’est **pas** automatiquement mise à jour lorsque la collection MongoDB sous-jacente évolue.

Cette asymétrie constitue un problème architectural central identifié durant le stage : **modifier MongoDB ne suffit pas pour mettre à jour BM25**. Concrètement, si un document est ajouté via `POST /upload_file` sans mécanisme correctif, le retrieveur dense retrouvera immédiatement les nouveaux chunks, tandis que le retrieveur BM25 continuera d’opérer sur son index figé au démarrage du service, ignorant purement et simplement les nouveaux contenus lors de la phase de recherche lexicale. À l’inverse, un document supprimé de MongoDB resterait indéfiniment indexé par BM25, pouvant conduire le pipeline à sélectionner un chunk fantôme, absent de la base persistante.

4.3.2 Solution retenue : `sync_retrievers()`

La solution adoptée pour ce problème est une fonction de resynchronisation explicite, invoquée systématiquement après toute opération de modification de la collection documentaire (ajout, suppression, mise à jour). Cette fonction reconstruit intégralement le retrieveur BM25 — ainsi que le retrieveur dense et l’ensemble hybride — à partir de l’état courant de MongoDB.

```
1 def sync_retrievers():
2     global bm25, hybrid, dense
3     latest_docs = []
4
```

```
5     for item in collection.find({}):
6         latest_docs.append(
7             Document(
8                 page_content=item.get("text", ""),
9                 metadata={
10                     "source": item.get("source", ""),
11                     "id": item.get("id")
12                 }
13             )
14         )
15
16     if latest_docs:
17         bm25 = BM25Retriever.from_documents(latest_docs)
18         bm25.k = 5
19         dense = vector_store.as_retriever(search_kwargs={"k": 5})
20         hybrid = EnsembleRetriever(
21             retrievers=[bm25, dense],
22             weights=[0.6, 0.4]
23         )
24     else:
25         bm25 = None
26         dense = None
27         hybrid = None
```

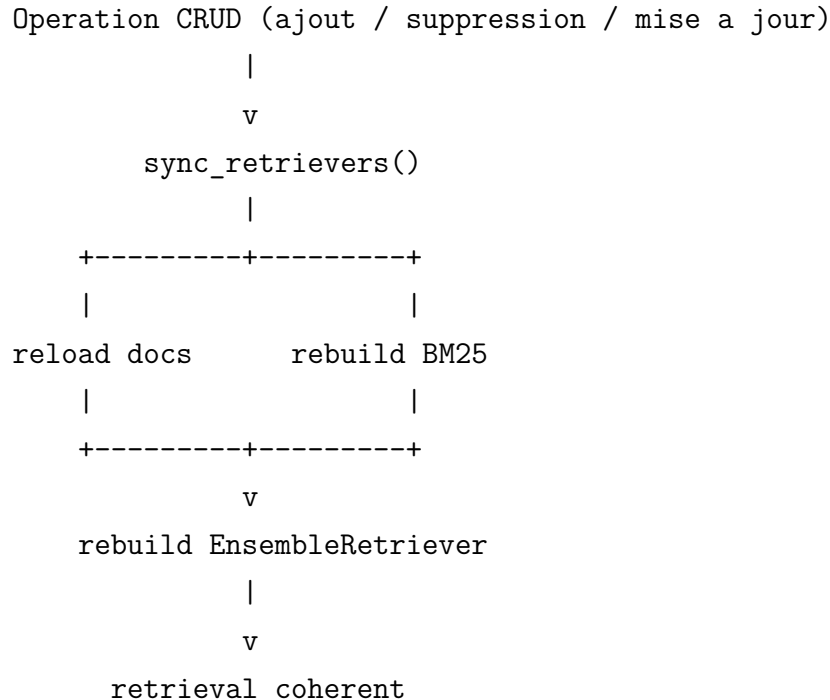
Listing 4.6 – Fonction de synchronisation des retrievers après modification de la base

Plusieurs éléments méritent une analyse approfondie :

1. **Utilisation du mot-clé `global`** : les variables `bm25`, `hybrid` et `dense` sont déclarées comme globales au module, afin que leur réaffectation à l’intérieur de la fonction soit visible depuis les autres parties du code, notamment l’endpoint `/ask` qui consomme directement le retriever `hybrid` lors du traitement d’une requête utilisateur.
2. **Rechargement intégral (*full reload*)** : la fonction ne réalise aucune mise à jour incrémentale ; elle recharge *l’intégralité* des documents présents dans la collection via `collection.find({})`, puis reconstruit BM25 depuis zéro. Cette approche, qualifiée de « brute-force » mais robuste, garantit la cohérence de l’index à chaque appel, au prix d’un coût de reconstruction proportionnel au nombre total de chunks — une limite explicitement identifiée pour le passage à l’échelle (cf. section 6.2.2).
3. **Gestion du cas de collection vide** : la branche `else` traite le cas particulier où la base ne contient plus aucun chunk (par exemple après suppression du dernier document), en réinitialisant les trois retrievers à `None` plutôt que de tenter de construire un `BM25Retriever` sur une liste vide, ce qui provoquerait une exception.

4.3.3 Flux de synchronisation

Le schéma logique de resynchronisation, déclenché après chaque opération CRUD, peut être résumé ainsi :



Cette logique permet au système de garantir la cohérence du retrieval hybride sans nécessiter de redémarrage manuel du service à chaque modification documentaire — exigence fonctionnelle centrale pour un backend destiné à une maintenance dynamique du corpus en production.

4.4 Gestion Dynamique du Cache lors des Opérations CRUD

4.4.1 Justification de l’invalidation systématique

Le mécanisme de cache Redis, détaillé au chapitre précédent, introduit un risque spécifique dans un contexte de base documentaire mutable : une réponse mise en cache *avant* une modification du corpus (ajout, suppression ou mise à jour d’un document) peut devenir **obsolète** après cette modification, tout en continuant à être servie aux utilisateurs tant que son TTL de 24 heures n’a pas expiré. Ce phénomène est particulièrement problématique pour un chatbot administratif, où l’exactitude et la fraîcheur de l’information sont des exigences de premier ordre.

La solution retenue consiste à invalider *globalement* le cache à chaque opération de modification de la base de connaissance, via une fonction `clear_cache()`, appelée immédiatement après `sync_retrievers()` dans les trois endpoints CRUD concernés.

4.4.2 Intégration dans les endpoints CRUD

Ajout d'un document (POST /upload_file)

L'ajout d'un nouveau document déclenche la séquence complète d'ingestion, suivie de la double étape de resynchronisation :

```
1 # 1. Verification de l'extension du fichier
2 # 2. Lecture du fichier en memoire
3 # 3. Selection du parser adapte (parse_docx / parse_pdf /
   parse_excel)
4 # 4. Creation des chunks (chunking + MD5)
5 # 5. Creation des objets Document LangChain
6 # 6. Insertion dans la collection MongoDB / vector store
7 # 7. Resynchronisation des retrievers
8 sync_retrievers()
9 # 8. Invalidation du cache
10 clear_cache()
```

Listing 4.7 – Séquence d'ajout d'un document et invalidation du cache

Suppression d'un document (DELETE /documents/{filename})

La suppression s'appuie sur le champ `source` des métadonnées pour cibler l'ensemble des chunks issus d'un même fichier :

```
1 result = collection.delete_many({"source": filename})
2
3 sync_retrievers()
4 clear_cache()
```

Listing 4.8 – Suppression d'un document et invalidation du cache

L'utilisation de `delete_many` plutôt que d'une suppression ciblée par identifiant unique illustre à nouveau le rôle pivot du champ `source` : la granularité de suppression retenue est le *document source*, et non le chunk individuel, ce qui est cohérent avec l'usage métier (un utilisateur souhaite retirer un document administratif obsolète dans son intégralité, non un fragment de celui-ci).

Mise à jour d'un document (PUT /document/{filename})

La mise à jour documentaire adopte une stratégie de **remplacement complet** plutôt qu'une logique de différence (*diff*) entre l'ancienne et la nouvelle version du document :

```
Ancien document
|
v
Suppression des chunks (source = filename)
|
v
Lecture du nouveau fichier
|
v
Extraction + chunking
|
v
Insertion des nouveaux chunks
|
v
sync_retrievers()
|
v
clear_cache()
```

Ce choix de conception — suppression intégrale suivie d’une réingestion complète — constitue un arbitrage délibéré en faveur de la simplicité d’implémentation. Une approche par différenciation (ne remplacer que les chunks effectivement modifiés) aurait nécessité une logique de comparaison sémantique ou syntaxique entre versions successives d’un document, dont la complexité n’était pas justifiée au regard de la volumétrie du corpus traité durant le stage. La contrepartie de ce choix est un coût de traitement légèrement supérieur lors d’une mise à jour mineure (l’intégralité du document est retraitée, même si une seule phrase a changé), coût jugé acceptable dans le contexte applicatif visé.

4.4.3 Synthèse du couplage CRUD – État

Le tableau 4.1 synthétise, pour chaque opération de gestion documentaire, les actions de resynchronisation déclenchées.

Cette systématisation — toute opération *mutative* déclenche invariablement les deux fonctions de resynchronisation — constitue un invariant architectural simple mais robuste, garantissant qu’aucune modification de la base documentaire ne peut laisser le système dans un état incohérent, que ce soit au niveau du retrieval (BM25 désynchronisé) ou au niveau de la couche applicative (cache Redis obsolète).

Opération	Endpoint	sync_retrievers()	clear_cache()
Ajout	POST /upload_file	Oui	Oui
Suppression	DELETE /documents/{filename}	Oui	Oui
Mise à jour	PUT /document/{filename}	Oui	Oui
Lecture	GET /document/{filename}	Non	Non

TABLE 4.1 – Actions de synchronisation associées à chaque opération CRUD documentaire

4.5 Conteneurisation et Déploiement Docker

4.5.1 Objectifs de la conteneurisation

La dernière étape du développement a consisté à emballer le backend au sein d'une image Docker, avec pour objectifs la reproductibilité de l'environnement d'exécution, l'isolation des dépendances système et applicatives, et la facilitation d'un déploiement ultérieur sur une infrastructure cloud.

4.5.2 Analyse du Dockerfile

Le Dockerfile de référence, situé dans `v25/retrieval/Dockerfile`, est reproduit ci-dessous :

```

1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --default-timeout=1000 --retries 10 --no-cache-dir
   -r requirements.txt
7
8 RUN python -c "from sentence_transformers import CrossEncoder;
   CrossEncoder('BAAI/bge-reranker-base')"
9 RUN python -c "from langchain_huggingface import
   HuggingFaceEmbeddings; HuggingFaceEmbeddings(model_name='
   intfloat/multilingual-e5-large')"
10
11 COPY . .
12
13 EXPOSE 8000
14 CMD ["uvicorn", "rag.py:app", "--host", "0.0.0.0", "--port", "8000"
   ]

```

Listing 4.9 – Dockerfile de conteneurisation du backend RAG

Choix de l'image de base

L'image `python:3.10-slim` constitue un compromis entre légèreté et complétude de l'environnement. Comparée à une image `python:3.10` standard, la variante `slim` exclut un ensemble de paquets systèmes non essentiels à l'exécution d'une application Python, réduisant ainsi la taille finale de l'image et, par extension, le temps de transfert lors du déploiement. Ce choix reste cependant compatible avec l'installation, via `pip`, des bibliothèques scientifiques lourdes requises par le projet (`sentence-transformers`, `langchain-huggingface`, `pandas`), qui embarquent leurs propres dépendances binaires.

Installation des dépendances avec tolérance aux erreurs réseau

La commande d'installation des dépendances Python intègre deux paramètres de robustesse :

```
1 pip install --default-timeout=1000 --retries 10 --no-cache-dir -r
   requirements.txt
```

Le paramètre `-default-timeout=1000` étend significativement le délai d'attente par défaut de `pip`, une précaution justifiée par la taille importante de certains paquets de l'écosystème (notamment `torch`, dépendance transitive de `sentence-transformers`), dont le téléchargement peut échouer prématurément avec un timeout par défaut sur une connexion réseau limitée. Le paramètre `-retries 10` complète cette tolérance en autorisant jusqu'à dix tentatives de récupération en cas d'échec transitoire. Enfin, `-no-cache-dir` désactive la mise en cache locale des paquets téléchargés par `pip`, ce qui réduit la taille finale des *layers* de l'image Docker, ce cache n'étant d'aucune utilité dans le contexte d'une image destinée à être construite une seule fois puis déployée telle quelle.

Pré-téléchargement des modèles Hugging Face au moment du *build*

L'élément le plus notable du Dockerfile réside dans les deux instructions `RUN` suivantes, exécutées *avant* la copie du code applicatif :

```
1 RUN python -c "from sentence_transformers import CrossEncoder; \
2 CrossEncoder('BAAI/bge-reranker-base') "
3
4 RUN python -c "from langchain_huggingface import
   HuggingFaceEmbeddings; \
5 HuggingFaceEmbeddings(model_name='intfloat/multilingual-e5-large') "
```

Ces deux lignesinstancient respectivement le modèle de reranking `BAAI/bge-reranker-base` et le modèle d'embeddings `intfloat/multilingual-e5-large`, ce qui a pour effet secondaire de déclencher leur téléchargement complet depuis le *Hub* Hugging

Face et leur mise en cache dans le système de fichiers de l'image, à l'emplacement par défaut utilisé par la bibliothèque `transformers` (généralement `~/.cache/huggingface`).

L'intérêt de cette approche est double :

- **Déplacement du coût de téléchargement vers la phase de construction** : sans ce mécanisme, le premier démarrage du conteneur en production déclencherait le téléchargement — potentiellement de plusieurs centaines de mégaoctets à plus d'un gigaoctet cumulé pour les deux modèles — directement au moment de l'initialisation du serveur Uvicorn, retardant d'autant la disponibilité du service et introduisant une dépendance réseau critique au démarrage.
- **Exploitation du système de *layer caching* Docker** : en isolant le téléchargement des modèles dans des instructions `RUN` *avant* la commande `COPY` . . . qui copie le code source applicatif, Docker peut réutiliser la *layer* contenant les modèles pré-téléchargés lors de reconstructions ultérieures de l'image, tant que le fichier `requirements.txt` n'a pas changé — même si le code de `rag.py` est modifié. Cette organisation de l'ordre des instructions n'est pas anodine : elle traduit une bonne pratique standard de construction d'images Docker, consistant à placer les étapes les plus coûteuses et les moins susceptibles de changer en amont du Dockerfile.

La contrepartie de cette stratégie est une augmentation de la taille finale de l'image (les poids des modèles étant embarqués directement dans celle-ci) et un temps de *build* initial plus long, ce qui constitue un arbitrage classique entre temps de construction et temps de démarrage — ce dernier étant jugé plus critique dans un contexte de déploiement où la disponibilité rapide du service prime.

Exposition du port et commande de démarrage

L'instruction `EXPOSE 8000` documente le port sur lequel le serveur ASGI est destiné à écouter, sans toutefois publier automatiquement ce port sur l'hôte — cette dernière opération relevant de la commande `docker run` (option `-p`) ou de la configuration d'orchestration utilisée en déploiement. La commande de démarrage :

```
1 CMD ["uvicorn", "rag.py:app", "--host", "0.0.0.0", "--port", "8000"]
```

lance le serveur Uvicorn en liant l'application FastAPI `app` définie dans `rag.py` sur l'ensemble des interfaces réseau du conteneur (`0.0.0.0`), condition nécessaire pour que le service soit accessible depuis l'extérieur du conteneur une fois le port publié.

4.5.3 Discussion

La stratégie de conteneurisation retenue privilégie la simplicité et la prévisibilité du temps de démarrage sur l'optimisation stricte de la taille de l'image. Cette approche

est cohérente avec le périmètre du stage, centré sur la validation fonctionnelle et la performance du pipeline RAG plutôt que sur l’optimisation fine d’une infrastructure de production à grande échelle. Des axes d’amélioration — non implémentés durant le stage — pourraient inclure l’utilisation d’un *build multi-stage* pour réduire la taille finale de l’image, ou l’externalisation des poids de modèles vers un volume monté plutôt qu’une intégration directe dans l’image, ces pistes s’inscrivant dans la continuité des axes de production et de sécurité identifiés en section 5.7 de la synthèse technique.

4.6 Conclusion du Chapitre

Ce chapitre a détaillé les mécanismes internes du pipeline d’ingestion et les solutions apportées à deux problèmes architecturaux centraux du système : la désynchronisation entre l’état persistant (MongoDB) et l’état en mémoire (BM25), résolue par la fonction `sync_retrievers()` ; et l’obsolescence potentielle des réponses mises en cache après modification du corpus, résolue par une invalidation systématique via `clear_cache()` à chaque opération CRUD. La stratégie de chunking fixe — blocs de 1000 mots sans chevauchement, identifiés par hachage MD5 — ainsi que le pré-téléchargement des modèles lors du *build* Docker complètent cette analyse, en mettant en évidence des choix d’ingénierie orientés vers la simplicité opérationnelle, parfois au prix de limites explicitement assumées et documentées comme axes d’amélioration futurs (chapitre 5).

Chapitre 5

Évaluation Expérimentale, Résultats et Analyse des Performances

5.1 Introduction et objectifs de l'évaluation

Ce chapitre présente l'évaluation expérimentale du backend RAG développé au cours du stage. L'objectif n'est pas de produire une mesure de performance universelle ou un benchmark public, mais de documenter les **résultats expérimentaux internes** obtenus sur le jeu de test de développement, tels qu'ils ont été mesurés au fil des itérations successives du projet (versions `v17`, `v18`, et configurations historiques antérieures). Ces mesures ont servi de base aux décisions d'ingénierie qui ont conduit à la configuration finale retenue dans `v25/retrieval/rag.py`.

L'évaluation s'articule autour de deux axes complémentaires :

- une évaluation de la **qualité** des réponses générées, au moyen du framework RAGAS ;
- une évaluation de la **latence**, décomposée par composant du pipeline (*expansion*, *retrieval*, *reranking*, *génération*).

Ces deux axes sont indissociables dans un système RAG : améliorer le rappel documentaire ou la fidélité des réponses entraîne systématiquement un surcoût en temps de traitement et en tokens consommés. L'ensemble de ce chapitre doit donc être lu comme une analyse de **compromis d'ingénierie** plutôt que comme une recherche d'un optimum absolu.

Avertissement méthodologique : l'ensemble des valeurs numériques présentées dans ce chapitre (scores RAGAS, temps de latence, nombre de tokens) sont issues de mesures réalisées sur un jeu de test de développement restreint, dans le cadre des expérimentations internes menées pendant le stage. Elles ne constituent ni une garantie de performance en production, ni une évaluation sur un benchmark standardisé public.

5.2 Protocole d'évaluation et métriques RAGAS

5.2.1 Choix du framework RAGAS

L'évaluation qualitative du pipeline s'appuie sur le framework **RAGAS** (Retrieval-Augmented Generation Assessment), qui propose un ensemble de métriques permettant d'évaluer séparément la qualité du *retrieval* et la qualité de la *génération*, sans nécessiter systématiquement une vérité terrain rédigée manuellement pour chaque question.

Quatre métriques ont été retenues pour l'évaluation du système :

Faithfulness (Fidélité) Mesure dans quelle proportion les affirmations contenues dans la réponse générée sont effectivement soutenues par le contexte récupéré. Une valeur proche de 1.0 indique que le LLM ne génère (quasiment) aucune information non justifiée par les chunks fournis, ce qui est directement corrélé à la consigne système “*Answer ONLY from the provided context*” imposée dans le prompt de génération.

Answer Relevancy (Pertinence de la réponse) Mesure à quel point la réponse générée est directement en adéquation avec la question posée, indépendamment de sa véracité factuelle. Une réponse peut être fidèle au contexte tout en étant partiellement hors sujet ; cette métrique capture cet aspect complémentaire.

Context Precision (Précision du contexte) Mesure la proportion de chunks récupérés qui sont effectivement pertinents pour répondre à la question, parmi l'ensemble des chunks transmis au LLM. Cette métrique est directement influencée par la qualité du *reranking* et par la politique de sélection *top-3*.

Context Recall (Rappel du contexte) Mesure la capacité du système à retrouver, parmi l'ensemble des chunks disponibles dans le corpus, ceux qui sont nécessaires pour répondre correctement à la question. Cette métrique dépend essentiellement de l'étape de *retrieval hybride* (BM25 + dense) et de la *query expansion*.

5.2.2 Résultats RAGAS documentés dans le README

Le tableau 5.1 présente les scores RAGAS rapportés dans le README du dépôt, obtenus pour une configuration considérée comme optimisée dans le cadre du développement, avec un temps de réponse moyen approximatif de **2 secondes**.

Ces résultats appellent plusieurs observations. La **Faithfulness** maximale (1.00) confirme l'efficacité du prompt de génération contraint, qui empêche le modèle GPT-OSS 120B de mobiliser des connaissances externes au contexte fourni. La **Context Precision** également maximale (1.00) traduit l'effet du *reranking Cross-Encoder* suivi

TABLE 5.1 – Scores RAGAS documentés dans le README (résultats expérimentaux internes, jeu de test de développement)

Métrique	Valeur indiquée
Faithfulness	1.00
Answer Relevancy	0.86
Context Precision	1.00
Context Recall	0.74

de la sélection stricte des trois meilleurs chunks, qui élimine efficacement le bruit documentaire avant la génération.

En revanche, le **Context Recall** (0.74) est la métrique la plus faible de cette évaluation. Elle indique qu'environ un quart de l'information théoriquement nécessaire pour répondre de façon exhaustive n'est pas systématiquement retrouvée par le pipeline de retrieval, ce qui constitue une limite identifiée du système et un axe d'amélioration prioritaire (cf. section 6.3).

Précision méthodologique : ces scores sont issus du jeu de test interne utilisé pendant le développement et peuvent varier significativement selon le dataset d'évaluation, la formulation des questions et la configuration du pipeline au moment de la mesure. Ils ne doivent pas être interprétés comme une performance certifiée ou reproductible sur un corpus administratif de plus grande échelle.

5.2.3 Résultats RAGAS d'une configuration historique intermédiaire

Une configuration antérieure, reposant sur une combinaison différente de composants (embeddings `multilingual-e5-small`, base vectorielle **ChromaDB** — étape de prototypage historique, cf. section 3.4 —, retrieval hybride, et reranker `bge-reranker-large`), avait produit les résultats présentés dans le tableau 5.2.

TABLE 5.2 – Scores RAGAS d'une configuration historique intermédiaire (mesure expérimentale, non représentative de v25)

Métrique	Résultat historique
Faithfulness	0.9796
Answer Relevancy	0.8954
Context Recall	1.0

Il est notable que cette configuration historique atteignait un **Context Recall parfait** (1.0), supérieur à celui rapporté dans la configuration finale documentée

par le README (0.74). Cette observation illustre un point central de la démarche expérimentale du stage : la configuration retenue in fine (v25) n’a pas été choisie sur le seul critère du meilleur score de rappel, mais sur un **compromis global** intégrant également la latence, le coût du reranker et la simplicité d’exploitation, ce compromis étant détaillé dans la section 5.4.

5.3 Profilage de la latence et décomposition temporelle

5.3.1 Méthodologie de mesure

Le endpoint `/ask` instrumente chaque requête et retourne, en plus de la réponse textuelle, les temps mesurés pour chacune des étapes du pipeline :

- `expansion_time` : durée de l’appel au LLM d’expansion (Qwen 3.6 27B) ;
- `retrieval_time` : durée combinée du retrieval BM25 et de la recherche vectorielle dense ;
- `reranker_time` : durée du passage des candidats dans le Cross-Encoder ;
- `llm_time` : durée de l’appel au LLM de génération finale (GPT-OSS 120B) ;
- `total_time` : durée cumulée de bout en bout.

Cette instrumentation a permis de comparer objectivement différentes versions du pipeline au cours du stage, et notamment d’isoler l’effet de l’ajout de la *query expansion* sur la latence globale.

5.3.2 Comparaison des versions v17 et v18

Le tableau 5.3 présente une comparaison directe de deux mesures expérimentales représentatives, réalisées sur deux configurations successives du pipeline : la version v17, sans query expansion, utilisant `multilingual-e5-large` et `BAAI/bge-reranker-base` ; et la version v18, avec query expansion activée et un reranker `BAAI/bge-reranker-v2-m3`.

Cette comparaison met en évidence un résultat expérimental majeur du stage : le passage de v17 à v18 entraîne une **multiplication par un facteur proche de 2,56** du temps total de traitement (de 3.742 s à 9.572 s), pour une augmentation de **près de 2,83 fois** le nombre de tokens consommés (de 462 à 1307 tokens).

On observe formellement que l’écart de temps total est cohérent avec la somme des écarts par composant :

$$\Delta t_{\text{total}} \approx \Delta t_{\text{expansion}} + \Delta t_{\text{retrieval}} + \Delta t_{\text{reranker}} + \Delta t_{\text{LLM}}$$

TABLE 5.3 – Comparaison des temps mesurés entre les versions **v17** (sans expansion) et **v18** (avec expansion), mesures expérimentales sur scénario de test

Composant	v17 (sans expansion)	v18 (avec expansion)	Écart
Expansion	–	0.967 s	+0.967 s
Retrieval	1.519 s	2.330 s	+0.811 s
Reranker	0.349 s	4.519 s	+4.170 s
LLM (génération)	1.873 s	1.756 s	−0.117 s
Temps total	3.742 s	9.572 s	+5.830 s
Tokens	462	1307	+845

$$5.830 \text{ s} \approx 0.967 + 0.811 + 4.170 - 0.117 = 5.831 \text{ s}$$

Le facteur explicatif dominant de cette dégradation n’est **pas** l’expansion elle-même (qui n’ajoute que 0.967s), mais bien le changement de reranker : le passage de **bge-reranker-base** à **bge-reranker-v2-m3** fait passer le temps de reranking de 0.349s à 4.519s, soit une multiplication par un facteur supérieur à 12. Cette observation a constitué un élément déterminant dans le choix final de conserver **BAAI/bge-reranker-base** comme reranker de production dans **v25**, ce point étant détaillé en section 5.4.1.

L’augmentation du retrieval (+0.811s) et surtout du nombre de tokens (quasi triplé) s’explique quant à elle directement par la query expansion : les deux requêtes alternatives générées par **Qwen 3.6 27B** déclenchent des recherches BM25 et vectorielles supplémentaires, et le contexte assemblé à partir de davantage de candidats potentiels alourdit le prompt final transmis au LLM de génération.

5.3.3 Décomposition proportionnelle de la latence

La figure conceptuelle ci-dessous synthétise, pour chaque version, la part relative de chaque composant dans le temps total mesuré :

TABLE 5.4 – Part relative de chaque composant dans le temps total (calculée à partir du tableau 5.3)

Composant	Part en v17 (%)	Part en v18 (%)
Expansion	–	10.1%
Retrieval	40.6%	24.3%
Reranker	9.3%	47.2%
LLM	50.1%	18.3%

Cette décomposition confirme que dans la version **v18**, le **reranker devient le goulot d’étranglement principal** du pipeline (près de la moitié du temps total), alors qu’en **v17** c’est la génération LLM qui dominait le budget temporel. Ce déplacement

du goulot d'étranglement d'un composant à un autre selon la configuration retenue constitue un enseignement méthodologique central de la phase d'expérimentation.

5.4 Analyse des compromis (Trade-offs)

5.4.1 Qualité du reranker versus latence

Plusieurs rerankers ont été comparés expérimentalement au cours du stage : BAAI/bge-reranker-large, une variante Cross-Encoder de type MiniLM, BAAI/bge-reranker-base, ainsi que BAAI/bge-reranker-v2-m3. Le tableau 5.5 synthétise les observations rapportées lors des expérimentations.

TABLE 5.5 – Synthèse comparative des rerankers testés (observations expérimentales qualitatives)

Reranker	Comportement observé	Statut dans le projet
bge-reranker-large	Qualité élevée (associé à Faithfulness 0.9796, tableau 5.2), latence de plusieurs secondes	Configuration historique intermédiaire
Cross-Encoder MiniLM	Latence réduite (temps total autour de 1.6 s), mais Faithfulness dégradée (≈ 0.8667)	Variante rapide testée, non retenue
bge-reranker-base	Compromis retenu : reranking mesuré à 0.349 s (v17)	Reranker final retenu dans v25
bge-reranker-v2-m3	Reranking mesuré à 4.519 s (v18), fort impact sur la latence totale	Expérimentation, non retenu en production

Cette comparaison illustre un arbitrage classique en ingénierie RAG : il n'existe pas de reranker *dominant* sur l'ensemble des critères simultanément. Le modèle MiniLM, bien que le plus rapide observé (temps total ≈ 1.6 s), a été écarté car sa Faithfulness (≈ 0.8667) restait sensiblement inférieure à celle obtenue avec des rerankers plus lourds. À l'inverse, bge-reranker-large et bge-reranker-v2-m3, bien qu'associés à de meilleurs indicateurs de qualité dans certains essais, imposaient une latence jugée excessive pour un usage conversationnel.

Le choix final de BAAI/bge-reranker-base dans v25 traduit donc une décision d'ingénierie explicite : privilégier un **point d'équilibre** entre qualité de reranking et réactivité perçue par l'utilisateur, plutôt que de maximiser un unique indicateur de

qualité. Cette décision est cohérente avec le constat établi en synthèse (section 5.6) selon lequel *“le choix des modèles doit être fait conjointement sur la qualité et la latence, et non sur un seul critère”*.

5.4.2 Impact de la Query Expansion sur les tokens et le rappel documentaire

La comparaison v17/v18 (tableau 5.3) permet de quantifier précisément le coût de la query expansion en termes de consommation de tokens :

$$\text{Ratio tokens} = \frac{1307}{462} \approx 2.83$$

Autrement dit, l’ajout de l’étape d’expansion — qui génère deux requêtes alternatives transmises séparément au retrieval hybride — provoque un **quasi-triplement** du nombre de tokens comptabilisés pour une requête donnée. Ce surcoût s’explique par deux effets cumulés :

1. l’appel supplémentaire au LLM d’expansion (Qwen 3.6 27B) consomme lui-même des tokens en entrée (prompt d’expansion en arabe, cf. section relative à l’architecture) et en sortie (les deux reformulations générées) ;
2. l’élargissement du nombre de requêtes envoyées au retrieval hybride augmente potentiellement la taille et la diversité de l’ensemble de candidats, ce qui peut se répercuter sur la taille du contexte final transmis au LLM de génération.

Du point de vue du **rappel documentaire**, le document de synthèse indique que l’expansion *“s’est montrée utile pour améliorer le recall sur certaines formulations”*, en particulier lorsque le vocabulaire employé par l’utilisateur diverge fortement de la terminologie administrative du corpus. Ce bénéfice n’a toutefois pas été isolé par une mesure RAGAS dédiée comparant strictement v17 et v18 sur le *Context Recall* ; l’analyse repose donc principalement sur l’observation qualitative des mesures de latence et de tokens disponibles.

Un point de vigilance spécifique a été identifié concernant les questions formulées en **Darija** : une expansion trop agressive peut modifier le sens de la question ou produire une reformulation administrative trop éloignée de l’intention initiale de l’utilisateur. Ce constat a conduit à traiter la query expansion comme une **optimisation à surveiller**, plutôt que comme une étape systématiquement bénéfique, ce qui motive directement l’axe d’amélioration consistant à rendre son déclenchement **conditionnel** (cf. section 6.3 et pistes d’amélioration du chapitre précédent).

5.4.3 Synthèse de l'arbitrage global

Le tableau 5.6 résume l'arbitrage global observé entre les principales configurations testées durant le stage.

TABLE 5.6 – Synthèse des compromis qualité / latence / coût token observés (mesures expérimentales internes)

Configuration	Temps total	Tokens	Qualité relative
v17 (sans expansion, reranker base)	3.742 s	462	Bonne, sans gain de recall additionnel
v18 (avec expansion, reranker v2-m3)	9.572 s	1307	Recall potentiellement amélioré, coût élevé
Config. historique (e5-small, ChromaDB, reranker large)	Plusieurs secondes (reranker)	–	Faithfulness 0.9796, Recall 1.0
Variante rapide (Cross-Encoder MiniLM)	≈ 1.6 s	–	Faithfulness ≈ 0.8667

Cette synthèse confirme qu'aucune configuration testée ne domine simultanément sur les trois axes (latence, coût en tokens, qualité). La configuration finale **v25** — combinant **multilingual-e5-large**, retrieval hybride pondéré 0.6/0.4, reranker **bge-reranker-base** et sélection stricte des trois meilleurs chunks — représente le point de compromis retenu à l'issue de ces expérimentations, privilégiant une latence maîtrisée et une Faithfulness élevée (1.00 selon le README) au prix d'un Context Recall perfectible (0.74).

5.5 Rôle de la sélection des trois meilleurs chunks (Top-3)

La politique consistant à ne transmettre que les **trois meilleurs chunks** au LLM de génération, après l'étape de reranking, constitue un choix architectural central dont l'impact se mesure sur plusieurs dimensions à la fois.

5.5.1 Justification technique

Formellement, le pipeline de sélection peut être résumé comme suit :

$$\text{candidats}_{\text{hybride}} \xrightarrow{\text{Cross-Encoder}} \text{scores} \xrightarrow{\text{tri décroissant}} \text{top-3}$$

où le score de chaque paire (question, chunk) est calculé par le Cross-Encoder BAAI/bge-reranker-base :

```

1 pairs = [(request.question, doc.page_content) for doc in results]
2 scores = reranker.predict(pairs)
3
4 ranked_docs = [
5     doc
6     for _, doc in sorted(
7         zip(scores, results),
8         key=lambda x: x[0],
9         reverse=True
10    )
11 ]
12
13 final_docs = ranked_docs[:3]
```

Listing 5.1 – Sélection finale des trois meilleurs chunks après reranking

Cette limitation à trois chunks répond à un double objectif :

1. **Maîtrise de la fenêtre de contexte** transmise au LLM GPT-OSS 120B : en limitant le nombre de passages injectés dans le `user_prompt`, le système réduit mécaniquement la taille du prompt final, ce qui a un effet direct sur le temps d'inférence (`llm_time`) et sur le nombre de `completion_tokens` facturés.
2. **Réduction du bruit documentaire** : un contexte contenant des chunks faiblement pertinents dilue l'attention du modèle génératif et augmente le risque que la réponse s'appuie sur une information secondaire ou hors sujet, ce qui dégraderait à la fois la *Context Precision* et, indirectement, la *Faithfulness*.

5.5.2 Lien avec les résultats mesurés

Le score de **Context Precision** maximal (1.00) rapporté dans le tableau 5.1 est directement corrélé à cette politique de sélection stricte : en ne conservant que les candidats les mieux classés par le Cross-Encoder, la proportion de chunks non pertinents effectivement transmis au LLM est minimisée par construction.

Cette politique a toutefois une contrepartie déjà identifiée : elle ne peut pas compenser une insuffisance en amont du pipeline de retrieval. Si l'information nécessaire à la réponse n'apparaît dans aucun des chunks initialement récupérés par le retrieval hybride, aucune

étape de reranking ni de limitation du contexte ne pourra la faire apparaître a posteriori. Ce constat rejoint directement la limite du *Context Recall* imparfait (0.74) observée dans le tableau 5.1, et souligne que l'amélioration de la qualité globale du système dépend davantage d'un renforcement du *retrieval* (par exemple via une expansion conditionnelle ou une fusion par rang de type RRF) que d'un ajustement supplémentaire du nombre de chunks retenus en sortie de reranking.

Des essais de **sélection dynamique** (2 ou 3 chunks selon la question) et de **compression de contexte** ont par ailleurs été explorés au cours du stage, mais la politique *top-3 fixe* a finalement été retenue dans v25 en raison de sa simplicité d'implémentation et de la stabilité des résultats qu'elle procure, au détriment d'une optimisation plus fine mais plus coûteuse à mettre en œuvre.

5.6 Synthèse du chapitre

L'évaluation expérimentale menée durant le stage met en évidence plusieurs enseignements majeurs, tous issus de **mesures internes réalisées sur le jeu de test de développement** :

- la configuration finale v25 obtient une Faithfulness et une Context Precision élevées (respectivement 1.00), au prix d'un Context Recall perfectible (0.74), identifié comme limite principale du système ;
- le passage d'une configuration sans expansion (v17) à une configuration avec expansion et reranker plus lourd (v18) multiplie le temps de traitement par un facteur ≈ 2.56 et la consommation de tokens par un facteur ≈ 2.83 ;
- le **reranker** constitue, selon la configuration, le principal facteur de variation de la latence globale, davantage que la query expansion elle-même ;
- le choix du reranker BAAI/bge-reranker-base dans v25 résulte d'un arbitrage explicite entre qualité et latence, plutôt que d'une recherche du score maximal sur une seule métrique ;
- la politique de sélection *top-3* après reranking permet de maîtriser efficacement la fenêtre de contexte et la latence de génération, mais ne peut compenser les limites du retrieval en amont.

Ces constats justifient les axes d'amélioration identifiés en fin de projet, notamment la mise en place d'une **query expansion conditionnelle**, l'exploration de rerankers multilingues dédiés, et l'évolution vers une fusion par rang de type **Reciprocal Rank Fusion (RRF)**, pistes qui seront développées dans le chapitre consacré au bilan et aux perspectives du projet.

Chapitre 6

Limites Apparentes, Retours d'Expérience et Perspectives d'Évolution

6.1 Introduction

L'objectif de ce chapitre est de porter un regard critique et distancié sur le système livré à l'issue du stage. Un système RAG en production ne se juge pas uniquement sur sa capacité à répondre correctement dans des conditions favorables, mais également sur sa robustesse face au passage à l'échelle, sur la maîtrise de ses coûts opérationnels (latence, tokens) et sur sa capacité à évoluer sans dette technique excessive. La version finale `v25/retrieval/rag.py` constitue un livrable fonctionnel et cohérent, mais elle comporte des limites assumées, résultant d'arbitrages effectués pour respecter le périmètre et le calendrier du stage. Ce chapitre recense ces limites, propose des pistes d'évolution argumentées, puis conclut sur un bilan personnel des compétences acquises.

6.2 Limites Techniques Identifiées

6.2.1 Un cache exact, non sémantique

Le mécanisme de cache implémenté repose sur une normalisation simple de la question (`strip()`, `lower()`) suivie d'un hachage MD5 servant de clé Redis :

```
1 normalized_question = request.question.strip().lower()
2 cache_key = (
3     f"rag_cache:{hashlib.md5(normalized_question.encode('utf-8')).
4    hexdigest()}"
5 )
```

Listing 6.1 – Construction de la clé de cache

Cette approche garantit un temps de résolution en $O(1)$ pour une requête strictement identique déjà traitée, mais elle présente une limite structurelle : deux formulations sémantiquement équivalentes mais lexicalement différentes ne produisent pas le même haché, et donc pas le même `cache_key`. Par exemple, les questions « *Comment obtenir un extrait d'acte de naissance ?* » et « *Quelle est la procédure pour avoir un extrait de naissance ?* » sont sémantiquement quasi identiques mais généreront systématiquement deux entrées de cache distinctes, et donc deux exécutions complètes du pipeline RAG (expansion, retrieval hybride, reranking, génération). Dans un contexte administratif où de nombreux usagers reformulent une même intention de façon hétérogène, le gain réel du cache exact reste donc probablement inférieur à son potentiel théorique.

6.2.2 Reconstruction synchrone et intégrale de l'index BM25

La fonction `sync_retrievers()` constitue une solution simple mais coûteuse au problème de désynchronisation entre le vector store MongoDB et l'index BM25 en mémoire. À chaque opération CRUD (ajout, suppression, mise à jour d'un document), l'intégralité des chunks est rechargée depuis MongoDB et l'index BM25 est reconstruit dans son ensemble :

```

1 def sync_retrievers():
2     global bm25, hybrid, dense
3     latest_docs = []
4     for item in collection.find({}):
5         latest_docs.append(Document(...))
6     if latest_docs:
7         bm25 = BM25Retriever.from_documents(latest_docs)
8     ...

```

Listing 6.2 – Reconstruction complète de l'index sparse

Cette stratégie a l'avantage de la simplicité et de la fiabilité : elle élimine tout risque d'incohérence partielle entre les deux index. Sa limite est de nature algorithmique : le coût de reconstruction croît linéairement, voire plus, avec le nombre total de chunks présents en base ($\mathcal{O}(N)$ au minimum pour la relecture, auquel s'ajoute le coût de tokenisation et d'indexation propre à BM25). Sur un corpus de quelques centaines à quelques milliers de chunks — taille représentative du corpus administratif traité durant le stage — cette reconstruction reste rapide et imperceptible pour l'utilisateur. Elle deviendrait en revanche un point de contention sérieux pour un corpus à l'échelle nationale, notamment si les opérations d'écriture (ajouts, mises à jour) devenaient

fréquentes et concurrentes, chaque opération bloquant potentiellement le retrieval pendant la phase de reconstruction.

6.2.3 Chunking fixe à 1000 mots sans chevauchement

La version finale découpe le texte extrait en blocs fixes de 1000 mots, sans mécanisme de chevauchement (*overlap*), contrairement aux expérimentations intermédiaires qui avaient testé un `RecursiveCharacterTextSplitter` avec un `chunk_overlap` de 150 caractères :

```
1 chunk_size = 1000
2 words = text.split()
3 for i in range(0, len(words), chunk_size):
4     chunk_text = " ".join(words[i:i+chunk_size])
```

Listing 6.3 – Découpage sans chevauchement dans la version finale

L'absence d'overlap introduit un risque identifié mais non résolu dans le livrable final : une information cohérente au plan sémantique (une définition, une procédure en plusieurs étapes, une condition d'éligibilité suivie de son exception) peut se trouver scindée exactement à la frontière entre deux chunks consécutifs. Dans ce cas, ni le chunk i ni le chunk $i + 1$, pris isolément, ne portent l'information complète, ce qui peut dégrader le rappel effectif même lorsque le retrieval identifie correctement la zone pertinente du document. Ce découpage purement quantitatif (nombre de mots) ne tient par ailleurs pas compte des frontières sémantiques naturelles du document (titres, paragraphes, listes), contrairement à des approches de *semantic chunking* plus coûteuses à mettre en œuvre.

6.2.4 Absence de citation des sources dans la réponse API

Le endpoint `POST /ask` retourne la réponse générée, les temps d'exécution par étape et la consommation de tokens, mais ne restitue pas explicitement à l'appelant la liste des chunks ou documents sources ayant servi à construire le contexte, alors même que ces métadonnées (`source`, `id`) sont disponibles en interne à chaque étape du pipeline. Cette limite est particulièrement sensible dans le cadre d'un chatbot administratif : l'absence de traçabilité des sources empêche l'utilisateur de vérifier l'origine réglementaire ou documentaire de la réponse fournie, ce qui constitue un frein à la confiance et à l'auditabilité du système, deux exigences généralement fortes dans un contexte de service public.

Limite	Cause technique	Impact principal
Cache non sémantique	Clé = MD5(question normalisée)	Taux de <i>cache hit</i> sous-optimal
Reconstruction BM25 intégrale	Index sparse maintenu en mémoire	Non passage à l’échelle sur gros corpus
Chunking sans overlap	Découpage fixe par 1000 mots	Risque de coupure d’information
Absence de citations	Métadonnées non exposées en sortie	Traçabilité et confiance limitées

TABLE 6.1 – Synthèse des limites techniques identifiées sur la version v25

6.2.5 Synthèse des limites

6.3 Axes d’Amélioration et Perspectives Techniques

6.3.1 Mise en place d’un cache sémantique

Une évolution naturelle du cache exact consiste à introduire un **cache sémantique**, où la clé de recherche n’est plus une chaîne normalisée mais l’embedding vectoriel de la question. Le principe consisterait à :

1. calculer l’embedding $\vec{q}$ de la nouvelle question via le modèle `intfloat/multilingual-e5-large`, déjà utilisé pour le retrieval dense ;
2. comparer $\vec{q}$ aux embeddings des questions déjà mises en cache via une similarité cosinus ;
3. considérer qu’il y a *cache hit* si $\text{sim}(\vec{q}, \vec{q}_{\text{cache}}) \geq \tau$, où τ est un seuil empirique (typiquement $\tau \approx 0.92-0.97$ selon la tolérance souhaitée).

Cette approche permettrait de traiter comme équivalentes des questions telles que « *documents nécessaires pour un passeport* » et « *pièces à fournir pour obtenir un passeport* ». Le compromis à arbitrer serait alors celui du seuil τ : un seuil trop bas expose à des réponses erronées servies pour des questions en réalité distinctes (faux positifs sémantiques), tandis qu’un seuil trop conservateur ramène le mécanisme à un cache quasi exact. L’infrastructure MongoDB Atlas Vector Search déjà en place pourrait directement héberger cet index de questions mises en cache, évitant l’introduction d’un composant supplémentaire.

6.3.2 Query expansion conditionnelle

L’étape 12 des expérimentations (version v18) a montré que l’ajout systématique de la query expansion faisait passer le temps total de traitement de l’ordre de 3.742 s (v17, sans expansion) à 9.572 s, et le volume de tokens consommés de 462 à 1307, soit un

facteur proche de 2.8. Ce coût est appliqué uniformément à *toutes* les requêtes en cache miss, y compris celles pour lesquelles la formulation initiale était déjà suffisamment proche du vocabulaire documentaire pour ne pas nécessiter de reformulation.

Une évolution consisterait à rendre cette étape conditionnelle, selon une logique de décision légère en amont du pipeline :

$$\text{expansion}(q) = \begin{cases} \text{oui} & \text{si } |q| < L_{\min} \text{ ou score de confiance retrieval} < \theta \\ \text{non} & \text{sinon} \end{cases} \quad (6.1)$$

où $|q|$ désigne la longueur (en tokens) de la question et θ un seuil de confiance calculé, par exemple, à partir du score du meilleur candidat obtenu par un premier passage de retrieval dense à faible coût. Ce mécanisme permettrait de restreindre l'usage de l'appel LLM supplémentaire aux cas réellement ambigus, réduisant ainsi la latence moyenne et le coût en tokens à l'échelle du trafic global.

6.3.3 Remplacement de la pondération linéaire par une fusion de rangs (RRF)

La fusion actuelle repose sur une combinaison pondérée linéaire des scores normalisés de BM25 et de la recherche dense, avec des poids fixés empiriquement à 0.6 pour le sparse et 0.4 pour le dense :

$$\text{score}_{\text{hybride}}(d) = 0.6 \cdot \text{score}_{\text{BM25}}(d) + 0.4 \cdot \text{score}_{\text{dense}}(d) \quad (6.2)$$

Cette approche présente une fragilité connue : les échelles de score de BM25 et de la similarité cosinus des embeddings ne sont pas naturellement comparables, ce qui rend la pondération sensible au corpus et potentiellement instable d'un domaine documentaire à l'autre. La **Reciprocal Rank Fusion (RRF)** constitue une alternative plus robuste, indépendante de l'échelle des scores, puisqu'elle ne considère que le *rang* de chaque document dans chacune des listes de résultats :

$$\text{RRF}(d) = \sum_{r \in \{\text{BM25}, \text{dense}\}} \frac{1}{k + \text{rang}_r(d)} \quad (6.3)$$

où k est une constante de lissage (typiquement $k = 60$ dans la littérature) évitant qu'un document classé premier dans une seule liste ne domine excessivement le score final. Cette méthode supprimerait la nécessité de recalibrer manuellement les poids 0.6/0.4 à chaque évolution du corpus ou des modèles d'embeddings, au prix d'une perte d'interprétabilité directe de la contribution relative de chaque retriever.

6.3.4 Vers un RAG agentique (Agentic RAG / Self-RAG)

Le pipeline actuel demeure strictement linéaire :

Expansion → Retrieval → Reranking → Génération

Cette architecture ne comporte aucune boucle de rétroaction : si le contexte récupéré après reranking est insuffisant pour répondre à la question, le système ne dispose d'aucun mécanisme pour le détecter et relancer une recherche corrective — il se contente d'appliquer la consigne du prompt système invitant le LLM à signaler l'insuffisance du contexte, ce qui reste une garde-fou au niveau de la génération et non une amélioration du retrieval lui-même.

Une évolution vers une architecture de type **Agentic RAG** ou **Self-RAG**, orchestrée par exemple via **LangGraph**, permettrait d'introduire un graphe d'états dans lequel le système pourrait :

1. évaluer, via un modèle léger ou une heuristique de score, si le contexte récupéré après reranking est suffisant pour répondre à la question posée ;
2. si le contexte est jugé insuffisant, reformuler automatiquement la requête (au-delà de la simple expansion actuelle, en tenant compte du contexte déjà récupéré et jugé incomplet) ;
3. relancer un cycle de retrieval avec la requête reformulée, potentiellement en élargissant le nombre de candidats interrogés ;
4. le cas échéant, orienter la requête vers un outil ou une source différente selon le type de question détecté (question factuelle simple, question procédurale multi-étapes, question hors périmètre du corpus).

Cette architecture transformerait le pipeline d'une chaîne figée en un agent capable d'auto-évaluation, au prix d'une complexité d'orchestration et d'une latence potentiellement accrue sur les cas nécessitant plusieurs itérations — un compromis qui devra être mesuré empiriquement, dans la continuité de la démarche d'itération expérimentale déjà engagée pendant le stage.

6.3.5 Renforcement de la sécurité applicative

La version livrée ne met pas en œuvre de couche d'authentification ou d'autorisation sur les endpoints de gestion documentaire (`/upload_file`, `/documents/{filename}`, `/document/{filename}`), ce qui constitue une limite significative en vue d'un déploiement en environnement réel. Les axes de renforcement identifiés sont les suivants :

- **Authentification par jeton JWT** : sécurisation des appels API via un jeton signé, porteur de l'identité de l'appelant et d'une durée de validité limitée, à valider par un middleware FastAPI en amont de chaque route sensible.

- **Autorisation par rôle (RBAC)** : distinction entre un rôle de consultation (accès à `/ask`, `/documents`) et un rôle d'administration documentaire (accès à `/upload_file`, `DELETE`, `PUT`), afin de restreindre les opérations de modification du corpus aux seuls utilisateurs habilités.
- **Limitation de débit (*rate limiting*)** : protection contre les abus et la saturation du pipeline, particulièrement critique pour les appels vers les modèles Groq (Qwen, GPT-OSS) dont le coût est directement proportionnel au volume de requêtes traitées.
- **Validation stricte des fichiers uploadés** : contrôle du type MIME réel, de la taille maximale et, le cas échéant, analyse antivirus des documents avant ingestion.
- **Journalisation et supervision** : traçabilité des accès et des opérations CRUD à des fins d'audit, en complément de l'instrumentation de latence déjà en place.

Ces éléments, cités comme piste d'amélioration dans la documentation du dépôt, n'ont pas été implémentés dans le cadre du stage mais constituent un prérequis incontournable avant toute mise en production du système sur un périmètre exposé publiquement.

6.4 Bilan Personnel et Synthèse des Acquis du Stage

Ce stage a permis de conduire, de bout en bout, la conception d'un système RAG complet, depuis les choix fondamentaux d'architecture jusqu'à la conteneurisation du livrable final. Plusieurs enseignements transversaux se dégagent de cette expérience.

Sur le plan technique, ce projet a permis d'acquérir une compréhension fine des compromis inhérents à tout système de recherche d'information hybride : l'articulation entre exactitude lexicale (BM25) et similarité sémantique (embeddings denses), l'arbitrage entre le rappel apporté par un retrieval large et la précision exigée par un reranking coûteux, ou encore la tension entre le gain de rappel potentiel de la query expansion et son coût en latence et en tokens — un compromis observé très concrètement entre les configurations `v17` et `v18`. Ces arbitrages ne relèvent pas de choix théoriques abstraits mais de décisions d'ingénierie mesurées empiriquement, ce qui a renforcé une approche de développement fondée sur l'expérimentation et l'évaluation systématique plutôt que sur l'intuition seule.

Sur le plan méthodologique, la structuration du dépôt en versions successives (`v0` à `v25`) a constitué un exercice concret de gestion itérative d'un projet de recherche appliquée, où chaque version répondait à une problématique identifiée dans la précédente : introduction du retrieval hybride, puis du reranking, puis de l'expansion de requête, puis du cache, puis de la persistance documentaire externalisée. Cette démarche a également mis en évidence l'importance de maintenir une cohérence entre documentation et code

livré, un point de vigilance identifié explicitement dans ce rapport (écart entre le README et l'architecture réelle de v25).

Enfin, ce travail a permis de développer une vision d'ensemble sur la construction d'un backend d'intelligence artificielle applicative, articulant des composants de nature hétérogène — modèles d'embeddings, moteur de recherche lexicale, base de données vectorielle, modèles de langage servis via une API externe, mécanisme de cache, couche API REST et conteneurisation — au sein d'une architecture cohérente et opérationnelle. Les limites identifiées dans ce chapitre ne doivent pas être lues comme des insuffisances du travail réalisé, mais comme le résultat naturel d'un périmètre de stage nécessairement borné, et comme la base d'une feuille de route claire pour les évolutions futures du système.